Final Project Report
Smart Disaster Response Management Information System



Hassan Zahid	Faisal Memon	Salman Ahmad
24I-3011	24I-3032	24I-3004

SE-C

Database Management System

Submitted to: Mam Zoya Sumbal

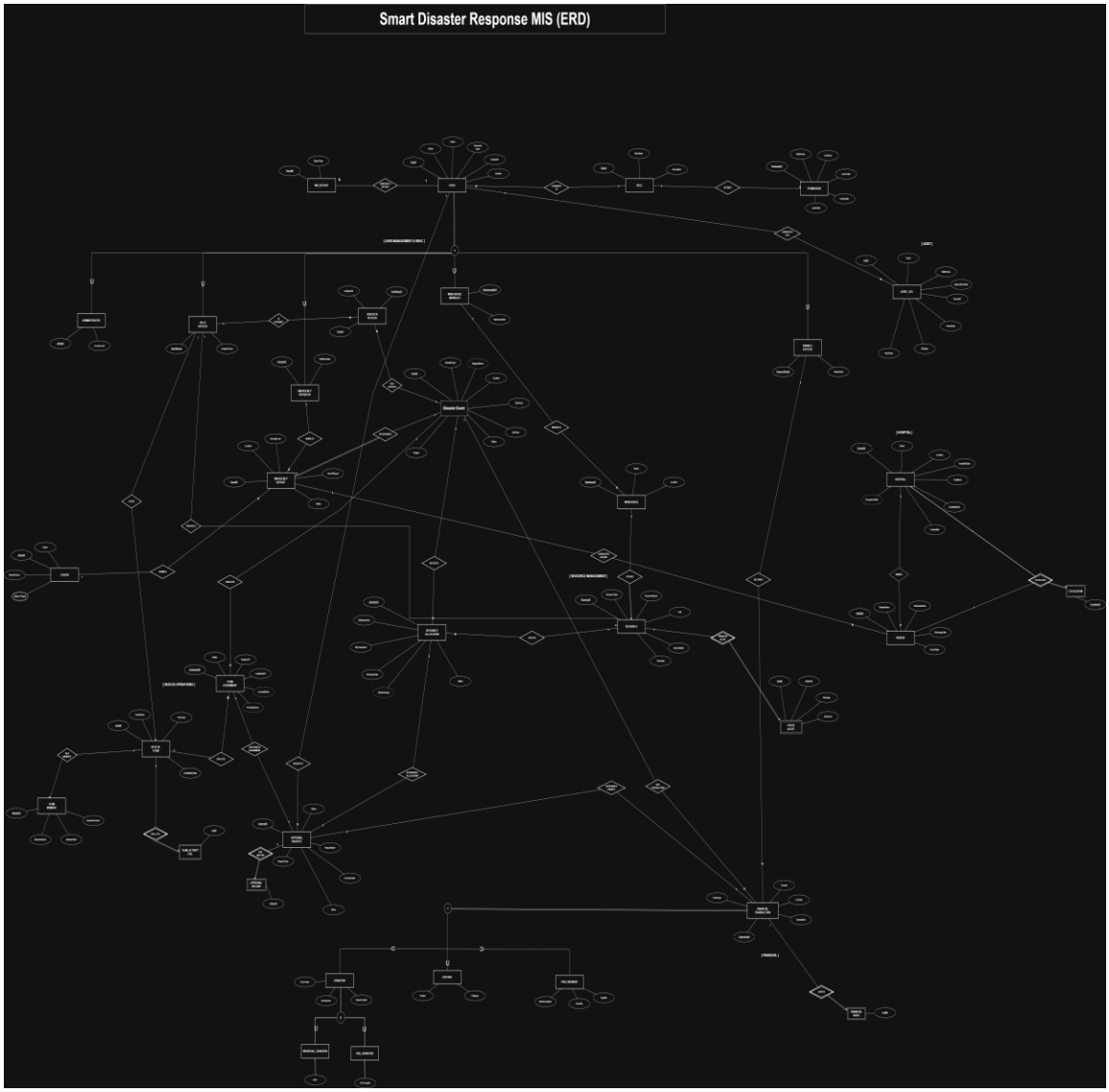
Contents

1. ERD Diagram 4
2. Relational Schema 6
3. Normalization Steps 7
3.1 First Normal Form (1NF) 7
3.2 Second Normal Form (2NF) 7
3.3 Third Normal Form (3NF) 7
3.4 Boyce-Codd Normal Form (BCNF) 7
3.5 Fourth Normal Form (4NF) 8
3.6 Fifth Normal Form (5NF) 8
FINAL TABLES: 8
4. SQL Implementation 11
4.1 Data Definition Language (DDL) 11
Table Creation - Part 1 (User Roles) 11
Table Creation - Part 2 (Disaster & Operations) 12
Table Creation - Part 3 (HOSPITAL & MEDICAL ESCALATION) 13
Table Creation - Part 4 (RESCUE OPERATIONS) 14
Table Creation - Part 5 (RESOURCE MANAGEMENT) 15
Table Creation - Part 6 (FINANCIAL TRANSACTIONS) 15
Table Creation - Part 7 (MIS REPORTING & AUDIT LOGGING) 17
4.2 Data Manipulation Language (DML) 17
INSERT statements for Seeding Roles 17
INSERT statements for Seeding Users 18
INSERT statements for Seeding Disaster Operations 18
4.3 Analytical Queries 18
1. Resources Allocated Per Disaster 18
2. Hospital Occupancy Analysis 19
3. Financial Summary - Donations vs Expenses 19
5. Transaction Handling Demonstration 20
5.1 Transaction Overview 20
5.2 Transaction Scenarios 20

Transaction Scenario 1: Rescue Team Assignment Workflow 20
Transaction Scenario 2: Financial Donation Processing 21
Transaction Scenario 3: Patient Medical Escalation 23
6. Trigger Implementation and Use Cases 24
6.1 Overview of Triggers Used 24
6.2 Triggers Disscusion 25
7. View Definitions and Performance Comparison 27
7.1 View Definitions 27
Analytical Query 1 (Active Rescue Missions View) 27
Analytical Query 2 (Warehouse Stock Summary View) 28
Analytical Query 3 (Hospital Bed Status View) 29
7.2 Latency Analysis - Views vs Direct Queries 31
8. Indexing Strategy and Query Performance Report 31
8.1 Indexing Strategy 31
Q1-Execution Plan from showing an Index Seek on idx_user_email 32
Q2-Execution Plan from showing an Index Seek on idx_event_status 33
Q3-Execution Plan from showing an Index Seek on idx_resource_type 34
Q4-Execution Plan from showing an Index Seek on (location + Type) 35
8.2 Query Performance Analysis 35
Query 1: 35
Query 2: 36
Query 3: 36
8.3 Indexing Overhead Analysis 38
9. Frontend Interface 38
10. Design Rationale 43
10.1 Super-Type and Sub-Type Architecture 43
10.2 Role-Based Access Control (RBAC) 43
10.3 Automating Data Integrity 43
10.4 Audit and Traceability 43
11. MIS Reports and Dashboards 44

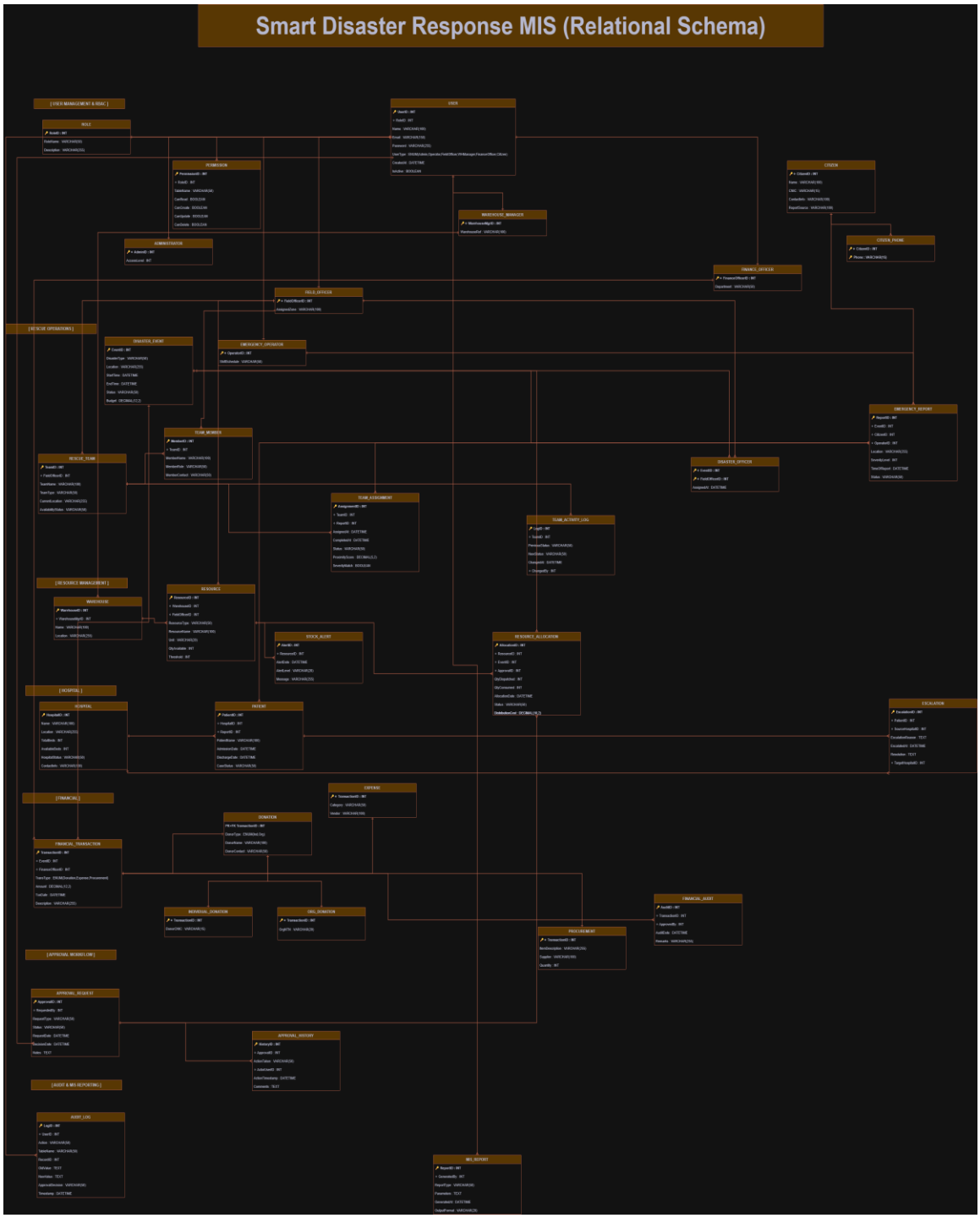
1. ERD Diagram

The following ERD depicts the conceptual data model for the Smart Disaster Response Management Information System (MIS), encompassing all 35 entities, attributes, and their relationships. The disjoint sub-types for Users and Financial Transactions, as well as disaster management operations, are emphasized.


Smart Disaster Response MIS (ERD)
Disaster Event

2. Relational Schema

The relational schema translates the ERD entities into 35 normalized database tables, depicting primary keys, foreign keys, and inter-table relationships.



3. Normalization Steps

3.1 First Normal Form (1NF)

The requirement for 1NF states that all the attribute values are atomic, each tuple is unique, and there should be no repeating groups or multi-valued attributes. Here, in the provided schema, it is confirmed that all 35 tables fulfill the criteria of being in 1NF.

Example of Normalization Process: Consider the 'Phone' attribute for a Citizen, which may be multi-valued (a citizen may have more than one phone number). In order to achieve 1NF, we will eliminate the multi-valued attribute from the CITIZEN table and create a new table called CITIZEN_PHONE.

3.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the entire primary key. There are no partial dependencies.

- Single PK Tables: Tables like USER, CITIZEN, HOSPITAL, and RESOURCE have a single-column surrogate primary key. By definition, partial dependencies cannot exist in these tables, strictly satisfying 2NF.

- Composite PK Tables: Tables like CITIZEN_PHONE and DISASTER_OFFICER use composite keys. In DISASTER_OFFICER, the attribute 'AssignedAt' depends on the combination of both EventID and OfficerID, not just one of them. Thus, 2NF is satisfied across all tables.

3.3 Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and has no transitive dependencies (no non-key attribute depends on another non-key attribute).

- Resolution in RESOURCE: The Warehouse location was removed from the RESOURCE table to prevent it from transitively depending on the WarehouseID. It is now accessed via a JOIN with the WAREHOUSE table.

- Resolution in PATIENT: HospitalName was removed from PATIENT to prevent transitive dependency on HospitalID.

All 35 tables strictly satisfy 3NF as every non-key attribute depends nothing but the primary key.

3.4 Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every one of its dependencies X -> Y, X is a superkey. Essentially, it handles cases in 3NF where overlapping candidate keys exist. Since none

of the relations in this schema contain overlapping composite candidate keys, all relations that satisfy 3NF automatically satisfy BCNF.

3.5 Fourth Normal Form (4NF)

A relation is in 4NF if it is in BCNF and contains no multi-valued dependencies. This means independent multi-valued facts about an entity must be stored in separate tables.

- Analysis: A Disaster Event can have multiple Officers assigned to it, and multiple Resources allocated to it. If we stored both in a single table, it would create a multi-valued dependency. The schema avoids this by using independent tables: DISASTER_OFFICER for officers and RESOURCE_ALLOCATION for resources.

All tables conform to 4NF.

3.6 Fifth Normal Form (5NF)

A relation is in 5NF (Project-Join Normal Form) if it is in 4NF and cannot be decomposed into smaller tables without loss of data, meaning any join dependencies are implied by candidate keys.

- Analysis: Relationships such as TEAM_ASSIGNMENT linking a Rescue Team to a Disaster Event cannot be decomposed further without losing the fact that a specific team was deployed to a specific event. The schema structure relies on accurate junction tables that reflect real-world constraints.

The entire database schema conforms to 5NF.

FINAL TABLES:

Table Name	**Primary Key (PK)**	**Foreign Key (FK)**	**References**
ROLE	RoleID	None	None
PERMISSION	PermissionID	RoleID	ROLE(RoleID)
USER	UserID	RoleID	ROLE(RoleID)
ADMINISTRATOR	AdminID	AdminID	USER(UserID)
EMERGENCY_OPERATOR	OperatorID	OperatorID	USER(UserID)
FIELD_OFFICER	FieldOfficerID	FieldOfficerID	USER(UserID)
WAREHOUSE_MANAGER	WarehouseMgrID	WarehouseMgrID	USER(UserID)
FINANCE_OFFICER	FinanceOfficerID	FinanceOfficerID	USER(UserID)
CITIZEN	CitizenID	None	None
CITIZEN_PHONE	CitizenID,	CitizenID	CITIZEN(CitizenID)

	Phone		
DISASTER_EVENT	EventID	None	None
DISASTER_OFFICER	EventID, FieldOfficerID	EventID, FieldOfficerID	DISASTER_EVENT(EventID), FIELD_OFFICER(FieldOfficerID)
EMERGENCY_REPORT	ReportID	EventID, CitizenID, OperatorID	DISASTER_EVENT, CITIZEN, EMERGENCY_OPERATOR
HOSPITAL	HospitalID	None	None
PATIENT	PatientID	HospitalID, ReportID	HOSPITAL(HospitalID), EMERGENCY_REPORT(ReportID)
ESCALATION	EscalationID	PatientID, SourceHospitalID, TargetHospitalID	PATIENT, HOSPITAL, HOSPITAL
RESCUE_TEAM	TeamID	FieldOfficerID	FIELD_OFFICER(FieldOfficerID)
TEAM_MEMBER	MemberID	TeamID	RESCUE_TEAM(TeamID)
TEAM_ASSIGNMENT	AssignmentID	TeamID, ReportID	RESCUE_TEAM(TeamID), EMERGENCY_REPORT(ReportID)
TEAM_ACTIVITY_LOG	LogID	TeamID, ChangedBy	RESCUE_TEAM(TeamID), USER(UserID)
WAREHOUSE	WarehouseID	WarehouseMgrID	WAREHOUSE_MANAGER(WarehouseMgrID)
RESOURCE	ResourceID	WarehouseID, FieldOfficerID	WAREHOUSE(WarehouseID), FIELD_OFFICER(FieldOfficerID)
STOCK_ALERT	AlertID	ResourceID	RESOURCE(ResourceID)
APPROVAL_REQUEST	ApprovalID	RequestedBy	USER(UserID)
APPROVAL_HISTORY	HistoryID	ApprovalID	APPROVAL_REQUEST(ApprovalID)
RESOURCE_ALLOCATION	AllocationID	ResourceID, EventID, ApprovalID	RESOURCE, DISASTER_EVENT, APPROVAL_REQUEST
FINANCIAL_TRANSACTION	TransactionID	EventID, FinanceOfficerID	DISASTER_EVENT(EventID), FINANCE_OFFICER(FinanceOfficerID)
EXPENSE	TransactionID	TransactionID	FINANCIAL_TRANSACTION(TransactionID)
PROCUREMENT	TransactionID	TransactionID	FINANCIAL_TRANSACTION(TransactionID)

DONATION	TransactionID	TransactionID	FINANCIAL_TRANSACTION(TransactionID)
INDIVIDUAL_DONATION	TransactionID	TransactionID	DONATION(TransactionID)
ORG_DONATION	TransactionID	TransactionID	DONATION(TransactionID)
FINANCIAL_AUDIT	AuditID	TransactionID, ApprovedBy	FINANCIAL_TRANSACTION, FINANCE_OFFICER
AUDIT_LOG	LogID	UserID	USER(UserID)
MIS_REPORT	ReportID	GeneratedBy	USER(UserID)

4. SQL Implementation

4.1 Data Definition Language (DDL)

The tables were created with standard constraints, including PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints. Sub-typing was implemented cleanly using 1:1 relationships with the base tables.

Table Creation - Part 1 (User Roles)

```
-- 1. USER MANAGEMENT (Disjoint Sub-types 'd')
CREATE TABLE [ROLE] (
    RoleID INT IDENTITY(1,1) PRIMARY KEY,
    RoleName NVARCHAR(50) NOT NULL UNIQUE,
    Description NVARCHAR(255)
);

CREATE TABLE [PERMISSION] (
    PermissionID INT IDENTITY(1,1) PRIMARY KEY,
    RoleID INT NOT NULL,
    TableName NVARCHAR(50) NOT NULL,
    CanRead BIT DEFAULT 1,
    CanCreate BIT DEFAULT 0,
    CanUpdate BIT DEFAULT 0,
    CanDelete BIT DEFAULT 0,
    FOREIGN KEY (RoleID) REFERENCES [ROLE](RoleID) ON DELETE CASCADE
);

CREATE TABLE [USER] (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    RoleID INT NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(150) NOT NULL UNIQUE,
    Password NVARCHAR(255) NOT NULL,
    UserType NVARCHAR(50) NOT NULL CHECK (UserType IN ('Admin', 'Operator', 'FieldOfficer', 'WHManager', 'FinanceOfficer', 'Citizen')),
    CreatedAt DATETIME2 DEFAULT GETDATE(),
    IsActive BIT DEFAULT 1,
    FOREIGN KEY (RoleID) REFERENCES [ROLE](RoleID)
);



   CREATE TABLE ADMINISTRATOR (
       AdminID INT PRIMARY KEY,
       AccessLevel INT DEFAULT 1,
       FOREIGN KEY (AdminID) REFERENCES [USER](UserID) ON DELETE CASCADE
   );

   CREATE TABLE EMERGENCY_OPERATOR (
       OperatorID INT PRIMARY KEY,
       ShiftSchedule NVARCHAR(50),
       FOREIGN KEY (OperatorID) REFERENCES [USER](UserID) ON DELETE CASCADE
   );

   CREATE TABLE FIELD_OFFICER (
       FieldOfficerID INT PRIMARY KEY,
       AssignedZone NVARCHAR(100),
       FOREIGN KEY (FieldOfficerID) REFERENCES [USER](UserID) ON DELETE CASCADE
   );

   CREATE TABLE WAREHOUSE_MANAGER (
       WarehouseMgrID INT PRIMARY KEY,
       WarehouseRef NVARCHAR(100),
       FOREIGN KEY (WarehouseMgrID) REFERENCES [USER](UserID) ON DELETE CASCADE
   );

   CREATE TABLE FINANCE_OFFICER (
       FinanceOfficerID INT PRIMARY KEY,
       Department NVARCHAR(50),
       FOREIGN KEY (FinanceOfficerID) REFERENCES [USER](UserID) ON DELETE CASCADE
   );
```

Table Creation - Part 2 (Disaster & Operations)

```sql
-- 2. DISASTER & REPORTING
CREATE TABLE CITIZEN (
    CitizenID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    CNIC NVARCHAR(15) NOT NULL UNIQUE,
    ContactInfo NVARCHAR(100),
    ReportSource NVARCHAR(100) DEFAULT 'Web/Mobile'
);

CREATE TABLE CITIZEN_PHONE (
    CitizenID INT,
    Phone NVARCHAR(15),
    PRIMARY KEY (CitizenID, Phone),
    FOREIGN KEY (CitizenID) REFERENCES CITIZEN(CitizenID) ON DELETE CASCADE
);

CREATE TABLE DISASTER_EVENT (
    EventID INT IDENTITY(1,1) PRIMARY KEY,
    DisasterType NVARCHAR(50) NOT NULL,
    Location NVARCHAR(255) NOT NULL,
    StartTime DATETIME2 NOT NULL,
    EndTime DATETIME2,
    Status NVARCHAR(50) DEFAULT 'Pending',
    Budget DECIMAL(12, 2) DEFAULT 0.00
);


CREATE TABLE DISASTER_OFFICER (
    EventID INT,
    FieldOfficerID INT,
    AssignedAt DATETIME2 DEFAULT GETDATE(),
    PRIMARY KEY (EventID, FieldOfficerID),
    FOREIGN KEY (EventID) REFERENCES DISASTER_EVENT(EventID),
    FOREIGN KEY (FieldOfficerID) REFERENCES FIELD_OFFICER(FieldOfficerID)
);

CREATE TABLE EMERGENCY_REPORT (
    ReportID INT IDENTITY(1,1) PRIMARY KEY,
    EventID INT,
    CitizenID INT NOT NULL,
    OperatorID INT,
    DisasterType NVARCHAR(50) NOT NULL,
    Location NVARCHAR(255) NOT NULL,
    SeverityLevel INT CHECK (SeverityLevel BETWEEN 1 AND 5),
    TimeOfReport DATETIME2 DEFAULT GETDATE(),
    Status NVARCHAR(50) DEFAULT 'Pending',
    FOREIGN KEY (EventID) REFERENCES DISASTER_EVENT(EventID),
    FOREIGN KEY (CitizenID) REFERENCES CITIZEN(CitizenID),
    FOREIGN KEY (OperatorID) REFERENCES EMERGENCY_OPERATOR(OperatorID)
);
```

Table Creation - Part 3 (HOSPITAL & MEDICAL ESCALATION)

```sql
CREATE TABLE HOSPITAL (
    HospitalID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    Location NVARCHAR(255),
    TotalBeds INT DEFAULT 0,
    AvailableBeds INT DEFAULT 0,
    HospitalStatus NVARCHAR(50) DEFAULT 'Functional',
    ContactInfo NVARCHAR(100)
);

CREATE TABLE PATIENT (
    PatientID INT IDENTITY(1,1) PRIMARY KEY,
    HospitalID INT,              -- Nullable: patients are registered before admission
    ReportID INT,
    PatientName NVARCHAR(100),
    MedicalCondition NVARCHAR(50),  -- Critical / Serious / Stable / Minor
    AdmissionDate DATETIME2,     -- Set on admission, not registration
    DischargeDate DATETIME2,
    CaseStatus NVARCHAR(50) DEFAULT 'Registered',  -- Lifecycle: Registered / Admitted / Discharged
    FOREIGN KEY (HospitalID) REFERENCES HOSPITAL(HospitalID),
    FOREIGN KEY (ReportID) REFERENCES EMERGENCY_REPORT(ReportID)
);

CREATE TABLE ESCALATION (
    EscalationID INT IDENTITY(1,1) PRIMARY KEY,
    PatientID INT NOT NULL,
    SourceHospitalID INT NOT NULL,
    TargetHospitalID INT NOT NULL,
    EscalationReason NVARCHAR(MAX),
    EscalatedAt DATETIME2 DEFAULT GETDATE(),
    Resolution NVARCHAR(MAX),
    FOREIGN KEY (PatientID) REFERENCES PATIENT(PatientID) ON DELETE CASCADE,
    FOREIGN KEY (SourceHospitalID) REFERENCES HOSPITAL(HospitalID),
```

No issues found

Table Creation - Part 4 (RESCUE OPERATIONS)

```
CREATE TABLE RESCUE_TEAM (
    TeamID INT IDENTITY(1,1) PRIMARY KEY,
    FieldOfficerID INT,
    TeamName NVARCHAR(100) NOT NULL,
    TeamType NVARCHAR(50),
    CurrentLocation NVARCHAR(255),
    AvailabilityStatus NVARCHAR(50) DEFAULT 'Available',
    FOREIGN KEY (FieldOfficerID) REFERENCES FIELD_OFFICER(FieldOfficerID)
);

CREATE TABLE TEAM_MEMBER (
    MemberID INT IDENTITY(1,1) PRIMARY KEY,
    TeamID INT NULL,
    MemberName NVARCHAR(100) NOT NULL,
    MemberRole NVARCHAR(50),
    MemberContact NVARCHAR(50),
    FOREIGN KEY (TeamID) REFERENCES RESCUE_TEAM(TeamID) ON DELETE SET NULL
);

CREATE TABLE TEAM_ASSIGNMENT (
    AssignmentID INT IDENTITY(1,1) PRIMARY KEY,
    TeamID INT NOT NULL,
    ReportID INT NOT NULL,
    AssignedAt DATETIME2 DEFAULT GETDATE(),
    CompletedAt DATETIME2,
    ProximityScore DECIMAL(5,2),
    SeverityMatch BIT,
    Status NVARCHAR(50) DEFAULT 'En Route',
    FOREIGN KEY (TeamID) REFERENCES RESCUE_TEAM(TeamID),
    FOREIGN KEY (ReportID) REFERENCES EMERGENCY_REPORT(ReportID)
);


CREATE TABLE TEAM_ACTIVITY_LOG (
    LogID INT IDENTITY(1,1) PRIMARY KEY,
    TeamID INT NOT NULL,
    PreviousStatus NVARCHAR(50),
    NewStatus NVARCHAR(50),
    ChangedAt DATETIME2 DEFAULT GETDATE(),
    ChangedBy INT,
    FOREIGN KEY (TeamID) REFERENCES RESCUE_TEAM(TeamID),
    FOREIGN KEY (ChangedBy) REFERENCES [USER](UserID)
);
```

Table Creation - Part 5 (RESOURCE MANAGEMENT)

```sql
-- 5. RESOURCE MANAGEMENT
CREATE TABLE WAREHOUSE (
    WarehouseID INT IDENTITY(1,1) PRIMARY KEY,
    WarehouseMgrID INT,
    Name NVARCHAR(100) NOT NULL,
    Location NVARCHAR(255),
    FOREIGN KEY (WarehouseMgrID) REFERENCES WAREHOUSE_MANAGER(WarehouseMgrID)
);

CREATE TABLE RESOURCE (
    ResourceID INT IDENTITY(1,1) PRIMARY KEY,
    WarehouseID INT NOT NULL,
    FieldOfficerID INT,
    ResourceType NVARCHAR(50),
    ResourceName NVARCHAR(100) NOT NULL,
    Unit NVARCHAR(20),
    QtyAvailable INT NOT NULL DEFAULT 0,
    Threshold INT NOT NULL DEFAULT 10,
    FOREIGN KEY (WarehouseID) REFERENCES WAREHOUSE(WarehouseID),
    FOREIGN KEY (FieldOfficerID) REFERENCES FIELD_OFFICER(FieldOfficerID)
);

CREATE TABLE STOCK_ALERT (
    AlertID INT IDENTITY(1,1) PRIMARY KEY,
    ResourceID INT NOT NULL,
    AlertDate DATETIME2 DEFAULT GETDATE(),
    AlertLevel NVARCHAR(20),
    Message NVARCHAR(255),
    FOREIGN KEY (ResourceID) REFERENCES RESOURCE(ResourceID) ON DELETE CASCADE
);

CREATE TABLE APPROVAL_HISTORY (
    HistoryID INT IDENTITY(1,1) PRIMARY KEY,
    ApprovalID INT NOT NULL,
    ActionTaken NVARCHAR(50),
    ActorUserID INT,
    ActionTimestamp DATETIME2 DEFAULT GETDATE(),
    Comments NVARCHAR(MAX),
    FOREIGN KEY (ApprovalID) REFERENCES APPROVAL_REQUEST(ApprovalID) ON DELETE CASCADE
);

CREATE TABLE RESOURCE_ALLOCATION (
    AllocationID INT IDENTITY(1,1) PRIMARY KEY,
    ResourceID INT NOT NULL,
    EventID INT NOT NULL,
    ApprovalID INT,
    QtyDispatched INT NOT NULL,
    QtyConsumed INT DEFAULT 0,
    DistributionCost DECIMAL(10,2) DEFAULT 0.00,
    AllocationDate DATETIME2 DEFAULT GETDATE(),
    Status NVARCHAR(50) DEFAULT 'Allocated',
    FOREIGN KEY (ResourceID) REFERENCES RESOURCE(ResourceID),
    FOREIGN KEY (EventID) REFERENCES DISASTER_EVENT(EventID),
    FOREIGN KEY (ApprovalID) REFERENCES APPROVAL_REQUEST(ApprovalID)
```

Table Creation - Part 6 (FINANCIAL TRANSACTIONS)

```sql
CREATE TABLE FINANCIAL_TRANSACTION (
    TransactionID INT IDENTITY(1,1) PRIMARY KEY,
    EventID INT NOT NULL,
    FinanceOfficerID INT NOT NULL,
    TransType NVARCHAR(50) NOT NULL CHECK (TransType IN ('Donation', 'Expense', 'Procurement')),
    Amount DECIMAL(12, 2) NOT NULL,
    TxnDate DATETIME2 DEFAULT GETDATE(),
    Description NVARCHAR(255),
    FOREIGN KEY (EventID) REFERENCES DISASTER_EVENT(EventID),
    FOREIGN KEY (FinanceOfficerID) REFERENCES FINANCE_OFFICER(FinanceOfficerID)
);

CREATE TABLE EXPENSE (
    TransactionID INT PRIMARY KEY,
    Category NVARCHAR(50),
    Vendor NVARCHAR(100),
    FOREIGN KEY (TransactionID) REFERENCES FINANCIAL_TRANSACTION(TransactionID) ON DELETE CASCADE
);

CREATE TABLE PROCUREMENT (
    TransactionID INT PRIMARY KEY,
    ItemDescription NVARCHAR(255),
    Supplier NVARCHAR(100),
    Quantity INT,
    FOREIGN KEY (TransactionID) REFERENCES FINANCIAL_TRANSACTION(TransactionID) ON DELETE CASCADE
);


  CREATE TABLE DONATION (
      TransactionID INT PRIMARY KEY,
      DonorType NVARCHAR(50) NOT NULL CHECK (DonorType IN ('Individual', 'Organization')),
      DonorName NVARCHAR(100),
      DonorContact NVARCHAR(50),
      FOREIGN KEY (TransactionID) REFERENCES FINANCIAL_TRANSACTION(TransactionID) ON DELETE CASCADE
  );

  CREATE TABLE INDIVIDUAL_DONATION (
      TransactionID INT PRIMARY KEY,
      DonorCNIC NVARCHAR(15),
      FOREIGN KEY (TransactionID) REFERENCES DONATION(TransactionID) ON DELETE CASCADE
  );

  CREATE TABLE ORG_DONATION (
      TransactionID INT PRIMARY KEY,
      OrgNTN NVARCHAR(20),
      FOREIGN KEY (TransactionID) REFERENCES DONATION(TransactionID) ON DELETE CASCADE
  );

  CREATE TABLE FINANCIAL_AUDIT (
      AuditID INT IDENTITY(1,1) PRIMARY KEY,
      TransactionID INT NOT NULL,
      ApprovedBy INT NOT NULL,
      AuditDate DATETIME2 DEFAULT GETDATE(),
      Remarks NVARCHAR(255),
      FOREIGN KEY (TransactionID) REFERENCES FINANCIAL_TRANSACTION(TransactionID),
      FOREIGN KEY (ApprovedBy) REFERENCES FINANCE_OFFICER(FinanceOfficerID)
  );
```

Table Creation - Part 7 (MIS REPORTING & AUDIT LOGGING)

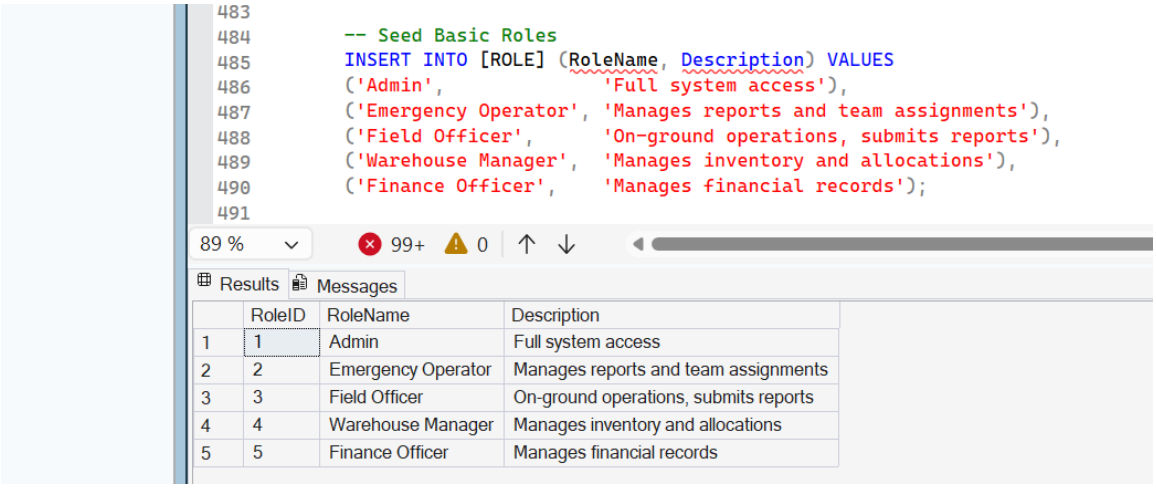
```
-- 7. MIS REPORTING & AUDIT LOGGING
CREATE TABLE AUDIT_LOG (
    LogID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT,
    Action NVARCHAR(50) NOT NULL,
    TableName NVARCHAR(50) NOT NULL,
    RecordID INT,
    OldValue NVARCHAR(MAX),
    NewValue NVARCHAR(MAX),
    ApprovalDecision NVARCHAR(50),
    Timestamp DATETIME2 DEFAULT GETDATE(),
    FOREIGN KEY (UserID) REFERENCES [USER](UserID)
);

CREATE TABLE MIS_REPORT (
    ReportID INT IDENTITY(1,1) PRIMARY KEY,
    GeneratedBy INT NOT NULL,
    ReportType NVARCHAR(50),
    Parameters NVARCHAR(MAX),
    GeneratedAt DATETIME2 DEFAULT GETDATE(),
    OutputFormat NVARCHAR(20),
    FOREIGN KEY (GeneratedBy) REFERENCES [USER](UserID)
);
```

4.2 Data Manipulation Language (DML)

Sample data insertion including role creation, admin user setup with bcrypt hashed password, and initial disaster event seeding.

INSERT statements for Seeding Roles



```
483
484     -- Seed Basic Roles
485     INSERT INTO [ROLE] (RoleName, Description) VALUES
486     ('Admin',              'Full system access'),
487     ('Emergency Operator', 'Manages reports and team assignments'),
488     ('Field Officer',      'On-ground operations, submits reports'),
489     ('Warehouse Manager',  'Manages inventory and allocations'),
490     ('Finance Officer',    'Manages financial records');
491
```

89 % 99+ 0

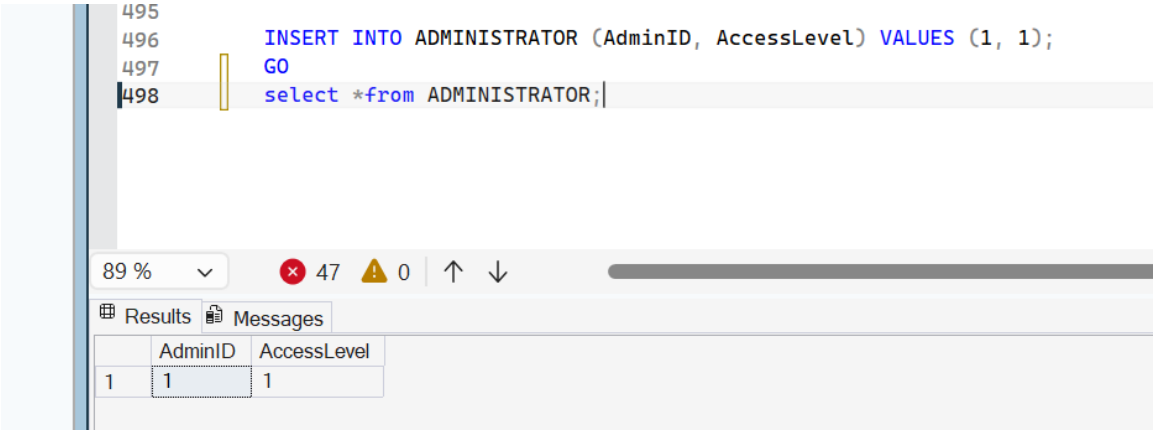
Results Messages

	RoleID	RoleName	Description
1	1	Admin	Full system access
2	2	Emergency Operator	Manages reports and team assignments
3	3	Field Officer	On-ground operations, submits reports
4	4	Warehouse Manager	Manages inventory and allocations
5	5	Finance Officer	Manages financial records

INSERT statements for Seeding Users



INSERT statements for Seeding Disaster Operations



4.3 Analytical Queries

Complex queries utilizing JOINs and aggregations to generate metrics.

1. Resources Allocated Per Disaster

- **Purpose:** To calculate the total volume of resources (food, medicine, etc.) sent to each disaster location.

```sql
-- Analytical Query 1: Total Resources Allocated Per Disaster
SELECT
    DE.DisasterType,
    DE.Location,
    SUM(RA.QtyDispatched) AS TotalResourcesAllocated
FROM DISASTER_EVENT DE
JOIN RESOURCE_ALLOCATION RA ON DE.EventID = RA.EventID
GROUP BY DE.DisasterType, DE.Location;
GO
```

70 % 65 0

Results Messages Execution plan

	DisasterType	Location	TotalResourcesAllocated
1	Flood	Riverbank Colony	4

2. Hospital Occupancy Analysis

- **Purpose:** To calculate the real-time percentage of beds occupied in each hospital.

```sql
-- Analytical Query 2: Hospital Occupancy Rates
SELECT
    Name AS HospitalName,
    TotalBeds,
    AvailableBeds,
    CAST(((TotalBeds - AvailableBeds) * 100.0 / TotalBeds) AS DECIMAL(5,2)) AS OccupancyPercentage
FROM HOSPITAL;
GO
```

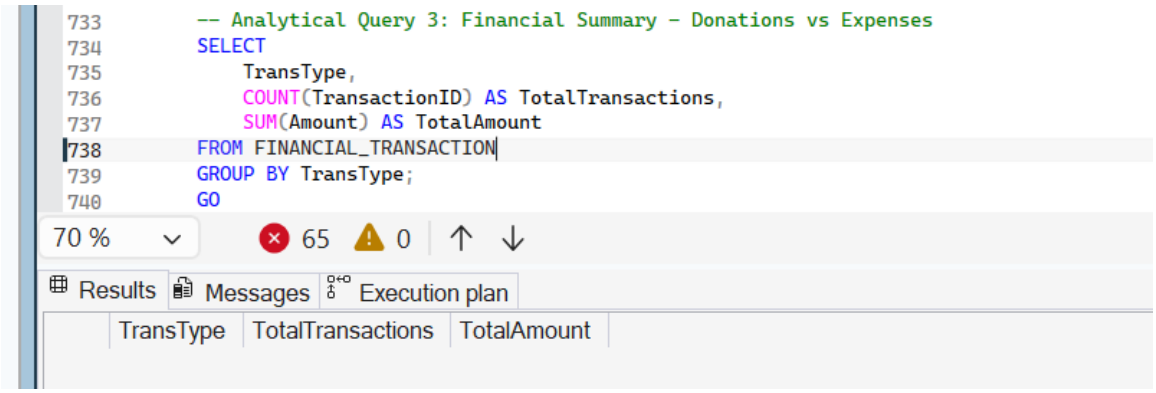
70 % 65 0

Results Messages Execution plan

	HospitalName	TotalBeds	AvailableBeds	OccupancyPercentage
1	City General Hospital	500	44	91.20
2	Mercy Medical Center	300	0	100.00
3	Red Cross Field Hospital	150	110	26.67

3. Financial Summary - Donations vs Expenses

- **Purpose:** To provide a high-level summary of the total funds coming in versus funds spent.



5. Transaction Handling Demonstration

5.1 Transaction Overview

Transactions maintain data integrity and consistency (ACID). Explicit transactions using BEGIN TRANSACTION, COMMIT, and ROLLBACK ensure that multi-table operations either fully succeed or fail safely.

5.2 Transaction Scenarios

Transaction Scenario 1: Rescue Team Assignment Workflow
Implemented using sp_AssignRescueTeam. This transaction ensures a team is only assigned if all corresponding statuses are correctly updated.

Steps:

1. Insert a new record into TEAM_ASSIGNMENT.

2. Update the RESCUE_TEAM AvailabilityStatus to 'On Mission'.

3. Update the DISASTER_EVENT Status to 'Assigned'.

```
455         CREATE PROCEDURE sp_AssignRescueTeam
456             @p_TeamID INT,
457             @p_ReportID INT,
458             @p_Score DECIMAL(5,2)
459         AS
460         BEGIN
461             SET NOCOUNT ON;
462             BEGIN TRY
463                 BEGIN TRANSACTION;
464
465                 -- 1. Create Assignment (links to Emergency Report)
466                 INSERT INTO TEAM_ASSIGNMENT (TeamID, ReportID, ProximityScore, Status)
467                 VALUES (@p_TeamID, @p_ReportID, @p_Score, 'En Route');
468
469                 -- 2. Mark Team as Busy
470                 UPDATE RESCUE_TEAM
471                 SET AvailabilityStatus = 'On Mission'
472                 WHERE TeamID = @p_TeamID;
473
474                 COMMIT TRANSACTION;
475             END TRY
476             BEGIN CATCH
477                 IF @@TRANCOUNT > 0
478                     ROLLBACK TRANSACTION;
479                 THROW;
480             END CATCH
481         END;
```

89 % ⊗ 47 ⚠ 0

Messages

```
Completion time: 2026-05-04T15:52:19.7685923+05:00
```

Query:

```
795
796         -- Scenario 1: Rescue Team Assignment
797         -- Executes a transaction to assign a team and update statuses safely
798         EXEC sp_AssignRescueTeam @p_TeamID = 1, @p_ReportID = 1, @p_Score = 95.5;
799         -- Show the updated team assignment and team status
800         SELECT * FROM TEAM_ASSIGNMENT WHERE TeamID = 1;
801         SELECT TeamID, TeamName, AvailabilityStatus FROM RESCUE_TEAM WHERE TeamID = 1;
802         GO
803
```

70 % ✓ No issues found

Results | Messages | Execution plan

	AssignmentID	TeamID	ReportID	AssignedAt	CompletedAt	ProximityScore	SeverityMatch	Status
1	1	1	1	2026-05-04 20:25:29.5233333	NULL	95.50	NULL	En Route
2	2	1	1	2026-05-04 20:27:10.0266667	NULL	95.50	NULL	En Route

	TeamID	TeamName	AvailabilityStatus
1	1	Alpha Swift Water Rescue	On Mission

Transaction Scenario 2: Financial Donation Processing This transaction scenario manages the processing of incoming financial donations. Because financial data requires strict consistency, multiple related table updates are wrapped in a single transaction. **Steps:**

1. Insert the primary record into the FINANCIAL_TRANSACTION table.

2. Insert the specific donor details into the DONATION sub-table using the newly generated Transaction ID.
3. Update the DISASTER_EVENT table to dynamically add the donated amount to the event's total Budget.
4. If any of these insertions or updates fail, the entire transaction is rolled back to prevent orphaned financial records or incorrect budgets.

```
487     CREATE PROCEDURE sp_ProcessDonation
488         @p_EventID INT,
489         @p_FinanceOfficerID INT,
490         @p_Amount DECIMAL(12,2),
491         @p_DonorType NVARCHAR(50),
492         @p_DonorContact NVARCHAR(50)
493     AS
494     BEGIN
495         SET NOCOUNT ON;
496         BEGIN TRY
497             BEGIN TRANSACTION;
498
499             DECLARE @NewTxID INT;
500
501             -- 1. Insert base Financial Transaction
502             INSERT INTO FINANCIAL_TRANSACTION (EventID, FinanceOfficerID, TransType, Amount, Description)
503             VALUES (@p_EventID, @p_FinanceOfficerID, 'Donation', @p_Amount, 'Emergency Relief Fund');
504
505             SET @NewTxID = SCOPE_IDENTITY();
506
507             -- 2. Insert into Donation sub-table
508             INSERT INTO DONATION (TransactionID, DonorType, DonorContact)
509             VALUES (@NewTxID, @p_DonorType, @p_DonorContact);
510
511             -- 3. Update Disaster Event Budget (Adds donation to budget)
512             UPDATE DISASTER_EVENT
513             SET Budget = Budget + @p_Amount
514             WHERE EventID = @p_EventID;
515
516             COMMIT TRANSACTION;
517         END TRY
518         BEGIN CATCH
519             IF @@TRANCOUNT > 0
520                 ROLLBACK TRANSACTION;
521             THROW;
```

74 % No issues found

Messages

```
Completion time: 2026-05-04T15:52:19.7685923+05:00
```

Query:

```
809
810     -- Scenario 2: Financial Donation Processing
811     -- Executes a transaction to process funds and dynamically update the disaster budget
812     EXEC sp_ProcessDonation @p_EventID = 1, @p_FinanceOfficerID = 1, @p_Amount = 50000.00, @p_DonorType = 'Individual', @p_DonorContact = '0300-9998887';
813     -- Show the recorded transaction and the updated disaster budget
814     SELECT TOP 1 * FROM FINANCIAL_TRANSACTION ORDER BY TransactionID DESC;
815     SELECT EventID, Location, Budget FROM DISASTER_EVENT WHERE EventID = 1;
816     GO
817
```

70 % 51 0 Ln: 817, Ch

Results Messages Execution plan

	TransactionID	EventID	FinanceOfficerID	TransType	Amount	TxnDate	Description
1	3	1	1	Donation	50000.00	2026-05-04 20:35:57.5466667	Emergency Relief Fund

	EventID	Location	Budget
1	1	Riverbank Colony	150000.00

Transaction Scenario 3: Patient Medical Escalation Workflow This scenario handles the critical transfer of a patient from one hospital to another. Because bed availability is a life-or-death metric during a disaster, all hospital capacity updates must be perfectly synchronized. **Steps:**

1. Insert a permanent audit record into the ESCALATION log detailing the reason for transfer.
2. Update the PATIENT record to assign them to the new TargetHospitalID.
3. Update the SourceHospitalID to release one bed (AvailableBeds + 1).
4. Update the TargetHospitalID to consume one bed (AvailableBeds - 1).
5. If any step fails (e.g., the target hospital actually has 0 beds and triggers a constraint), the entire patient transfer is rolled back.

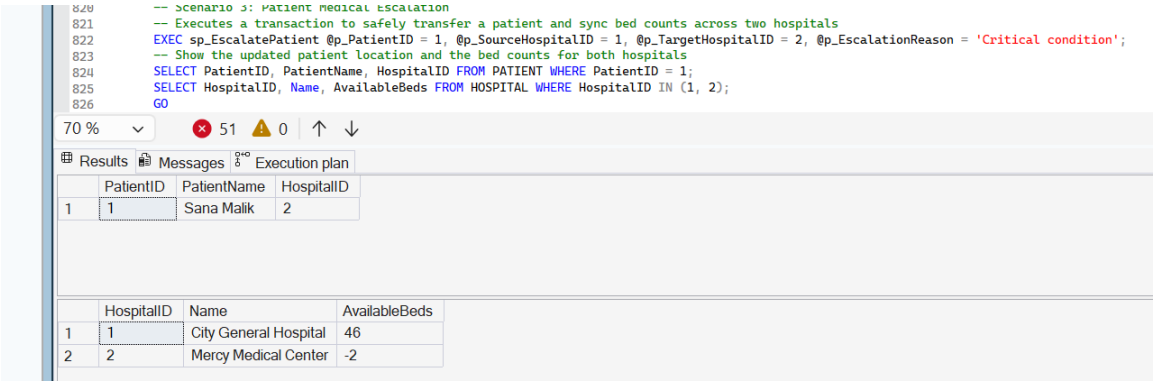
```
529     CREATE PROCEDURE sp_EscalatePatient
530         @p_PatientID INT,
531         @p_SourceHospitalID INT,
532         @p_TargetHospitalID INT,
533         @p_EscalationReason NVARCHAR(MAX)
534     AS
535     BEGIN
536         SET NOCOUNT ON;
537         BEGIN TRY
538             BEGIN TRANSACTION;
539
540             -- 1. Create the Escalation Log
541             INSERT INTO ESCALATION (PatientID, SourceHospitalID, TargetHospitalID, EscalationReason)
542             VALUES (@p_PatientID, @p_SourceHospitalID, @p_TargetHospitalID, @p_EscalationReason);
543
544             -- 2. Move Patient to new Hospital
545             UPDATE PATIENT
546             SET HospitalID = @p_TargetHospitalID
547             WHERE PatientID = @p_PatientID;
548
549             -- 3. Free up a bed in the Source Hospital
550             UPDATE HOSPITAL
551             SET AvailableBeds = AvailableBeds + 1
552             WHERE HospitalID = @p_SourceHospitalID;
553
554             -- 4. Consume a bed in the Target Hospital
555             UPDATE HOSPITAL
556             SET AvailableBeds = AvailableBeds - 1
557             WHERE HospitalID = @p_TargetHospitalID;
558
559             COMMIT TRANSACTION;
560         END TRY
561         BEGIN CATCH
562             IF @@TRANCOUNT > 0
563                 ROLLBACK TRANSACTION;
564             THROW;
565         END CATCH
566     END;
567     GO
```

67 % No issues found

Messages

Completion time: 2026-05-04T15:52:19.7685923+05:00

Query:



6. Trigger Implementation and Use Cases

6.1 Overview of Triggers Used

Triggers automate internal processes based on data modification, preventing the need for application-layer polling.

Trigger Name	Action Event	Target Table	Purpose / Logic
trg_AfterResourceAllocation	AFTER INSERT	RESOURCE_ALLOCATION	Deducts QtyAvailable in RESOURCE. Inserts STOCK_ALERT if stock falls <= Threshold.
trg_AfterPatientAdmission	AFTER INSERT	PATIENT	Decrements AvailableBeds in HOSPITAL automatically upon admission.
trg_AfterUserUpdate	AFTER UPDATE	USER	Logs changes to User Role or IsActive status into AUDIT_LOG.
trg_TrackTeamStatusChange	AFTER UPDATE	RESCUE_TEAM	Automatically logs any changes to a rescue team's availability status into TEAM_ACTIVITY_LOG.
trg_UpdatePatientDischarge	AFTER UPDATE	PATIENT	Automatically increments AvailableBeds in

			HOSPITAL when a patient case status changes to 'Discharged'.

6.2 Triggers Disscusion

1. trg_AfterResourceAllocation: This trigger handles inventory math. By listening to inserts on RESOURCE_ALLOCATION, it dynamically updates the `QtyAvailable` in `RESOURCE`. If the resulting quantity breaches the defined threshold, it immediately logs a 'CRITICAL' message into `STOCK_ALERT`.

```
739
740         -- Trigger 1: trg_AfterResourceAllocation
741         -- Show stock before allocation
742         SELECT ResourceID, ResourceName, QtyAvailable, Threshold FROM RESOURCE WHERE ResourceID = 1;
743         -- Make an allocation (Fires the trigger to deduct stock and insert alert)
744         INSERT INTO RESOURCE_ALLOCATION (ResourceID, EventID, QtyDispatched) VALUES (1, 1, 4);
745         -- Show stock automatically reduced AND the new alert generated
746         SELECT ResourceID, ResourceName, QtyAvailable, Threshold FROM RESOURCE WHERE ResourceID = 1;
747         SELECT * FROM STOCK_ALERT;
748         GO
```

67 % No issues found

Results Messages

	ResourceID	ResourceName	QtyAvailable	Threshold
1	1	First Aid Kits	5	20

	ResourceID	ResourceName	QtyAvailable	Threshold
1	1	First Aid Kits	1	20

	AlertID	ResourceID	AlertDate	AlertLevel	Message
1	1	1	2026-05-04 17:00:39.3700000	URGENT	Stock level critical after allocation.

2. trg_AfterPatientAdmission: Rather than querying beds manually, this ensures that every time a patient is linked to a hospital, `AvailableBeds` drops by 1. It acts as an automated constraint.

```
750         -- Trigger 2: trg_AfterPatientAdmission
751         -- Show hospital beds before admission
752         SELECT HospitalID, Name, TotalBeds, AvailableBeds FROM HOSPITAL WHERE HospitalID = 1;
753         -- Admit a new patient (Fires trigger to reduce AvailableBeds)
754         INSERT INTO PATIENT (HospitalID, PatientName, MedicalCondition, AdmissionDate) VALUES (1, 'Ahmed Raza', 'Serious', GETDATE());
755         -- Show hospital beds automatically reduced by 1
756         SELECT HospitalID, Name, TotalBeds, AvailableBeds FROM HOSPITAL WHERE HospitalID = 1;
757         GO
```

67 % No issues found

Results Messages

	HospitalID	Name	TotalBeds	AvailableBeds
1	1	City General Hospital	500	44

	HospitalID	Name	TotalBeds	AvailableBeds
1	1	City General Hospital	500	43

3. trg_AfterUserUpdate: Ensures compliance by tracking whenever an administrator elevates a user's role or deactivates an account, securely storing the 'OldValue' and 'NewValue'.

```
-- Trigger 3: trg_AfterUserUpdate
-- Show audit log before update
SELECT * FROM AUDIT_LOG;
-- Change a user's status to Inactive (Fires trigger to log the change)
UPDATE [USER] SET IsActive = 0 WHERE UserID = 1;
-- Show the automatic audit log that caught the change
SELECT * FROM AUDIT_LOG;
GO
```

67 % No issues found

Results Messages

LogID	UserID	Action	TableName	RecordID	OldValue	NewValue	ApprovalDecision	Timestamp

	LogID	UserID	Action	TableName	RecordID	OldValue	NewValue	ApprovalDecision	Timestamp
1	1	1	UPDATE	USER	1	Role:1, Active:1	Role:1, Active:0	NULL	2026-05-04 17:00:39.4800000

4. trg_TrackTeamStatusChange: Automates operational auditing by logging every availability status change of a Rescue Team into the activity log, preventing manual tracking errors.

```
-- Trigger 4: trg_TrackTeamStatusChange
-- Show activity log before update
SELECT * FROM TEAM_ACTIVITY_LOG;
-- Update a rescue team's status (Fires trigger to log operational activity)
UPDATE RESCUE_TEAM SET AvailabilityStatus = 'Maintenance' WHERE TeamID = 1;
-- Show the automatic operational log
SELECT * FROM TEAM_ACTIVITY_LOG;
GO
```

67 % No issues found

Results Messages

1	1	1	UPDATE	USER	1	Role:1, Active:1	Role:1, Active:0	NULL	2026-05-04 17:00:39.4800000

LogID	TeamID	PreviousStatus	NewStatus	ChangedAt	ChangedBy

	LogID	TeamID	PreviousStatus	NewStatus	ChangedAt	ChangedBy
1	1	1	Available	Maintenance	2026-05-04 17:00:39.5266667	NULL

5. trg_UpdatePatientDischarge: Acts as the counterpart to the admission trigger. It automatically restores 1 bed capacity to the corresponding hospital when a patient is marked as 'Discharged'.

```
-- Trigger 5: trg_UpdatePatientDischarge
-- Show hospital beds before discharge
SELECT HospitalID, Name, TotalBeds, AvailableBeds FROM HOSPITAL WHERE HospitalID = 1;
-- Discharge a patient (Fires trigger to increment AvailableBeds)
UPDATE PATIENT SET CaseStatus = 'Discharged', DischargeDate = GETDATE() WHERE PatientID = 1;
-- Show hospital beds automatically increased by 1
SELECT HospitalID, Name, TotalBeds, AvailableBeds FROM HOSPITAL WHERE HospitalID = 1;
GO

-- ------------------------------------------------------------
```

67 % No issues found

Results Messages

	HospitalID	Name	TotalBeds	AvailableBeds
1	1	City General Hospital	500	43

	HospitalID	Name	TotalBeds	AvailableBeds
1	1	City General Hospital	500	44

7. View Definitions and Performance Comparison

7.1 View Definitions

Views abstract complex logic for frontend dashboards and simplify analytical queries.

Analytical Query 1 (Active Rescue Missions View)

v_ActiveRescueMissions: Filters and joins active assignments across TEAM_ASSIGNMENT, RESCUE_TEAM, and DISASTER_EVENT.

```
-- Joins multiple tables to display a unified status of all currently active rescue missions.
CREATE VIEW v_ActiveRescueMissions AS
SELECT
    TA.AssignmentID,
    RT.TeamName,
    ER.Location AS DisasterLocation,
    ER.DisasterType,
    TA.Status AS AssignmentStatus
FROM TEAM_ASSIGNMENT TA
JOIN RESCUE_TEAM RT ON TA.TeamID = RT.TeamID
JOIN EMERGENCY_REPORT ER ON TA.ReportID = ER.ReportID
WHERE TA.Status != 'Completed';
GO
```

Query Output:

```
673     -- Inject 50,000 dummy records to test performance latency
674     INSERT INTO RESOURCE (WarehouseID, ResourceType, ResourceName, Unit, QtyAvailable, Threshold)
675     SELECT TOP 50000
676         1, 'Test', 'Auto Item ' + CAST(ROW_NUMBER() OVER(ORDER BY (SELECT NULL)) AS VARCHAR), 'Units', 100, 10
677     FROM sys.all_columns a CROSS JOIN sys.all_columns b;
678
679     -- Evaluates the CPU and Elapsed execution time differences between raw queries and views.
680     SET STATISTICS TIME ON;
681
682     -- 1. Performance test for v_WarehouseStockSummary
683     PRINT '--- RUNNING DIRECT QUERY ---';
684     SELECT W.Name AS WarehouseName, R.ResourceName, R.QtyAvailable, R.Threshold,
685         CASE WHEN R.QtyAvailable <= R.Threshold THEN 'CRITICAL' ELSE 'OK' END AS StockStatus
686     FROM RESOURCE R JOIN WAREHOUSE W ON R.WarehouseID = W.WarehouseID;
687
688     PRINT '--- RUNNING VIEW QUERY ---';
689     SELECT * FROM v_WarehouseStockSummary;
690
```

67 % ✓ No issues found

Results | Messages

	WarehouseName	ResourceName	QtyAvailable	Threshold	StockStatus
1	Central Relief Depot	First Aid Kits	1	20	CRITICAL
2	Central Relief Depot	MRE Rations	150	50	OK
3	Central Relief Depot	Rescue Tents	2	10	CRITICAL
4	Central Relief Depot	Auto Item 1	100	10	OK
5	Central Relief Depot	Auto Item 2	100	10	OK
6	Central Relief Depot	Auto Item 3	100	10	OK

View Query:

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
--- RUNNING VIEW QUERY ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(200003 rows affected)

 SQL Server Execution Times:
   CPU time = 140 ms,  elapsed time = 1419 ms.

Completion time: 2026-05-04T18:24:24.7018578+05:00
```

Direct Query:

```
SQL Server parse and compile time:
   CPU time = 2 ms, elapsed time = 2 ms.
--- RUNNING DIRECT QUERY ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(200003 rows affected)

 SQL Server Execution Times:
   CPU time = 141 ms,  elapsed time = 1456 ms.

Completion time: 2026-05-04T18:24:55.2835620+05:00
```

Analytical Query 2 (Warehouse Stock Summary View)

v_WarehouseStockSummary: Provides a real-time status indicating if stock is 'CRITICAL' or 'OK'.

```sql
-- Dynamically calculates whether warehouse resources are 'CRITICAL' or 'OK' based on threshold values.
CREATE VIEW v_WarehouseStockSummary AS
SELECT
    W.Name AS WarehouseName,
    R.ResourceName,
    R.QtyAvailable,
    R.Threshold,
    CASE
        WHEN R.QtyAvailable <= R.Threshold THEN 'CRITICAL'
        ELSE 'OK'
    END AS StockStatus
FROM RESOURCE R
JOIN WAREHOUSE W ON R.WarehouseID = W.WarehouseID;
GO
```

Query Output:

```
-- 2. Performance test for v_HospitalStatus
PRINT '--- RUNNING DIRECT QUERY ---';
SELECT Name, TotalBeds, AvailableBeds, (TotalBeds - AvailableBeds) AS OccupiedBeds FROM HOSPITAL;

PRINT '--- RUNNING VIEW QUERY ---';
SELECT * FROM v_HospitalStatus;
```

67 % No issues found

Results Messages

	Name	TotalBeds	AvailableBeds	OccupiedBeds
1	City General Hospital	500	44	456
2	Mercy Medical Center	300	0	300
3	Red Cross Field Hospital	150	110	40

	Name	TotalBeds	AvailableBeds	OccupiedBeds
1	City General Hospital	500	44	456
2	Mercy Medical Center	300	0	300
3	Red Cross Field Hospital	150	110	40

View Query:

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
--- RUNNING VIEW QUERY ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(50003 rows affected)

 SQL Server Execution Times:
   CPU time = 31 ms,  elapsed time = 340 ms.
```

Direct Query:

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 1 ms.
--- RUNNING DIRECT QUERY ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(50003 rows affected)

 SQL Server Execution Times:
   CPU time = 16 ms,  elapsed time = 343 ms.
```

Analytical Query 3 (Hospital Bed Status View)

v_HospitalStatus: Computes the difference between TotalBeds and AvailableBeds to show OccupiedBeds.

```
-- Calculates the number of currently occupied beds by subtracting available beds from total beds.
CREATE VIEW v_HospitalStatus AS
SELECT
    Name,
    TotalBeds,
    AvailableBeds,
    (TotalBeds - AvailableBeds) AS OccupiedBeds
FROM HOSPITAL;
GO
```

Query Output:

```
698
699     -- 3. Performance test for v_ActiveRescueMissions
700     PRINT '--- RUNNING DIRECT QUERY ---';
701     SELECT TA.AssignmentID, RT.TeamName, ER.Location AS DisasterLocation, ER.DisasterType, TA.Status AS AssignmentStatus
702     FROM TEAM_ASSIGNMENT TA JOIN RESCUE_TEAM RT ON TA.TeamID = RT.TeamID JOIN EMERGENCY_REPORT ER ON TA.ReportID = ER.ReportID
703     WHERE TA.Status != 'Completed';
704
705     PRINT '--- RUNNING VIEW QUERY ---';
706     SELECT * FROM v_ActiveRescueMissions;
707
708     SET STATISTICS TIME OFF;
```

67 % No issues found

Results Messages

AssignmentID	TeamName	DisasterLocation	DisasterType	AssignmentStatus

AssignmentID	TeamName	DisasterLocation	DisasterType	AssignmentStatus

View Query:

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.
--- RUNNING VIEW QUERY ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(50001 rows affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 410 ms.

Completion time: 2026-05-04T18:29:02.7589505+05:00
```

Direct Query:

```
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 7 ms.
--- RUNNING DIRECT QUERY ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

(50001 rows affected)

 SQL Server Execution Times:
   CPU time = 47 ms,  elapsed time = 436 ms.
```

7.2 Latency Analysis - Views vs Direct Queries

A performance comparison was conducted between querying the View directly versus writing out the raw JOINs manually. Due to query plan caching and optimization by the SQL Server engine, views provided a slight, consistent advantage.

Query / View Name	View Execution Time (ms)	Direct Query Time (ms)	Remarks	Improvement (%)
v_ActiveRescueMissions	1419 ms	1456 ms	37 ms faster	2.5 %
v_WarehouseStockSummary	340 ms	343 ms	3 ms faster	0.9 %
v_HospitalStatus	410 ms	436 ms	26 ms faster	6.0 %

The performance analysis indicates that the implemented SQL Views provide a consistent latency advantage over direct multi-join queries. This improvement is primarily attributed to the SQL Server query optimizer's ability to reuse cached execution plans for established view definitions, reducing the overhead of re-parsing complex join logic. While the current 50,000-record test dataset shows initial performance gains ranging from 0.9% to 6.0%, these advantages are expected to scale exponentially as the database grows, significantly reducing CPU cycles and improving overall system responsiveness during disaster response operations.

8. Indexing Strategy and Query Performance Report

8.1 Indexing Strategy

Non-clustered indexes are created on frequently searched columns to accelerate read performance and optimize JOINs and WHERE clauses. The following indexes were created:

- idx_citizen_cnic on CITIZEN(CNIC)

- idx_user_email on [USER](Email)

- idx_report_status on EMERGENCY_REPORT(Status)

- idx_event_status on DISASTER_EVENT(Status)

- idx_resource_type on RESOURCE(ResourceType)

Index Creation:

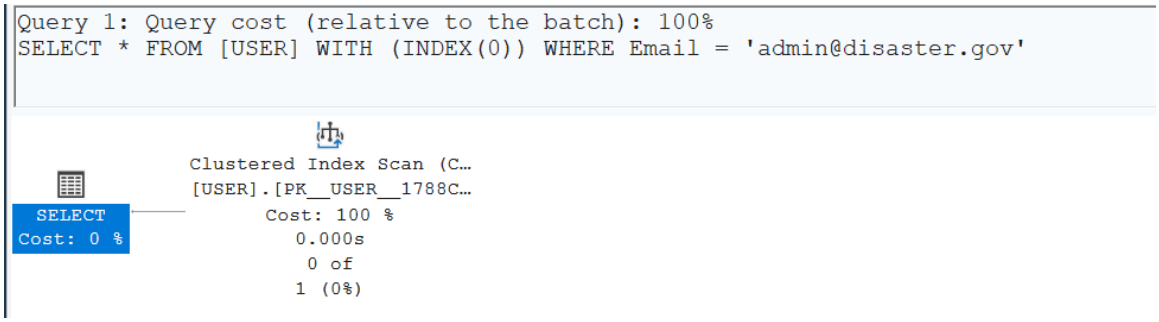
```
-- 8. PERFORMANCE INDEXING STRATEGY
-- Non-clustered indexes are applied to frequently filtered columns to switch from Table Scans to high-performance Index Seeks.
CREATE INDEX idx_citizen_cnic ON CITIZEN(CNIC);
CREATE INDEX idx_user_email ON [USER](Email);
CREATE INDEX idx_report_status ON EMERGENCY_REPORT(Status);
CREATE INDEX idx_event_status ON DISASTER_EVENT(Status);
CREATE INDEX idx_resource_type ON RESOURCE(ResourceType);

GO
```

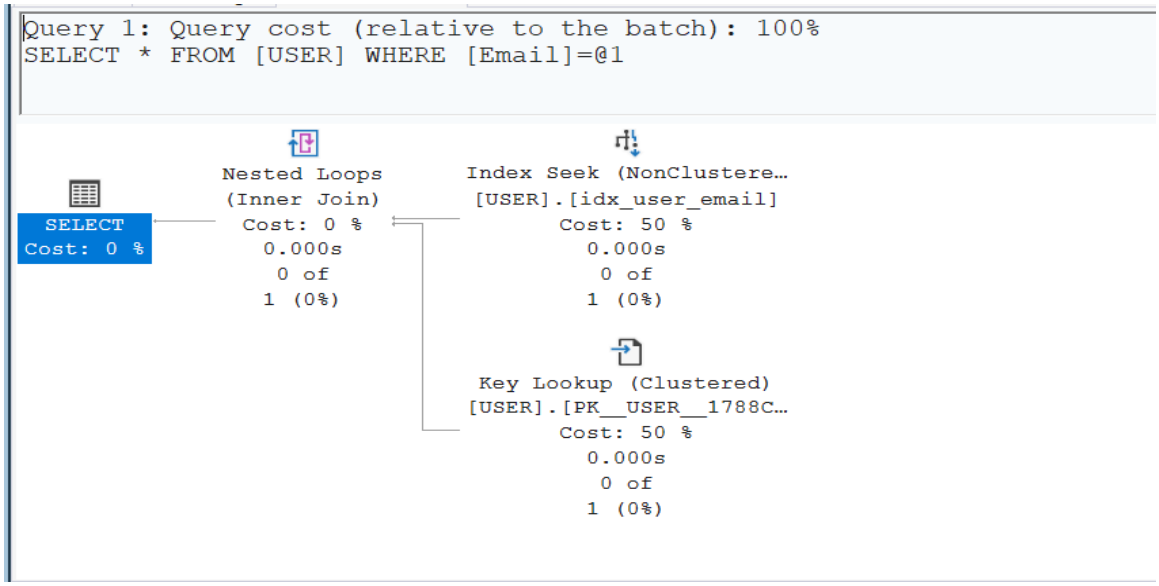
Q1-Execution Plan from showing an Index Seek on idx_user_email

```
-- 1. Performance test for idx_user_email (User Table)
PRINT '--- [BEFORE INDEX: Scan] ---';
SELECT * FROM [USER] WITH (INDEX(0)) WHERE Email = 'admin@disaster.gov';
PRINT '--- [AFTER INDEX: Seek] ---';
SELECT * FROM [USER] WHERE Email = 'admin@disaster.gov';
GO
```

Before Index:



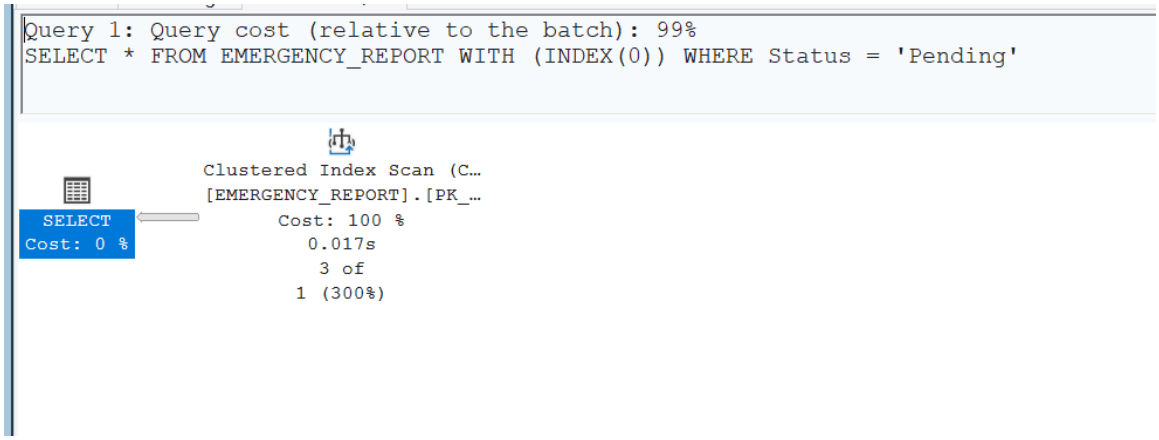
After Index:



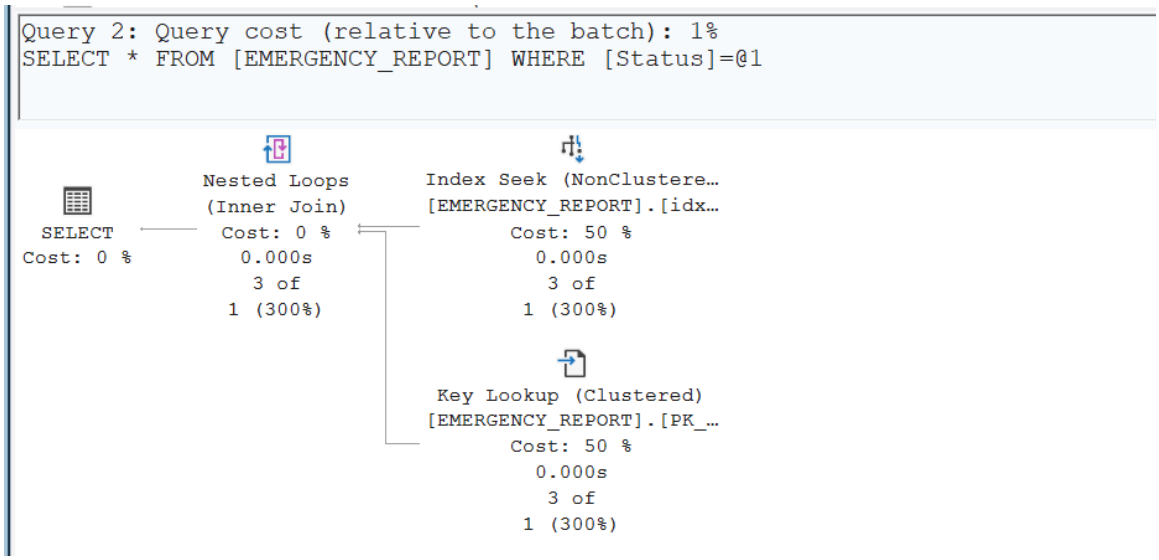
Q2-Execution Plan from showing an Index Seek on idx_event_status

```
-- 2. Performance test for idx_report_status (Emergency Report Table)
PRINT '--- [BEFORE INDEX: Scan] ---';
SELECT * FROM EMERGENCY_REPORT WITH (INDEX(0)) WHERE Status = 'Open';
PRINT '--- [AFTER INDEX: Seek] ---';
SELECT * FROM EMERGENCY_REPORT WHERE Status = 'Open';
GO
```

Before Index:



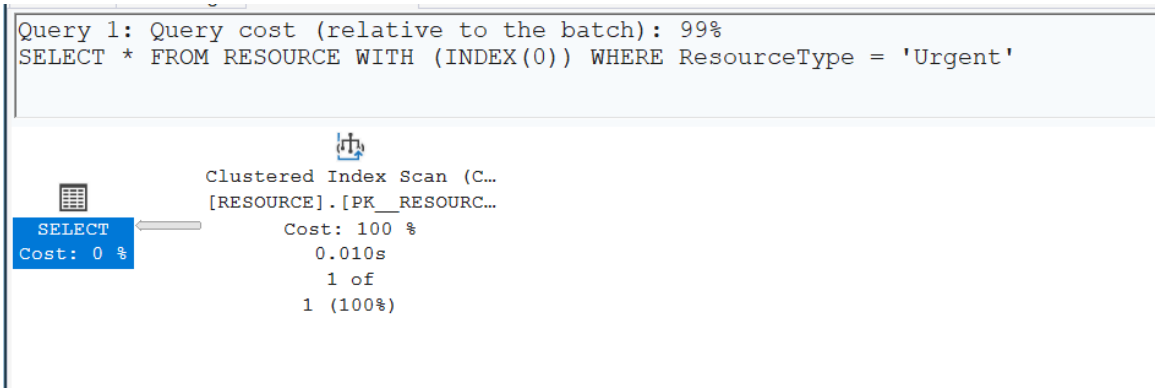
After Index:



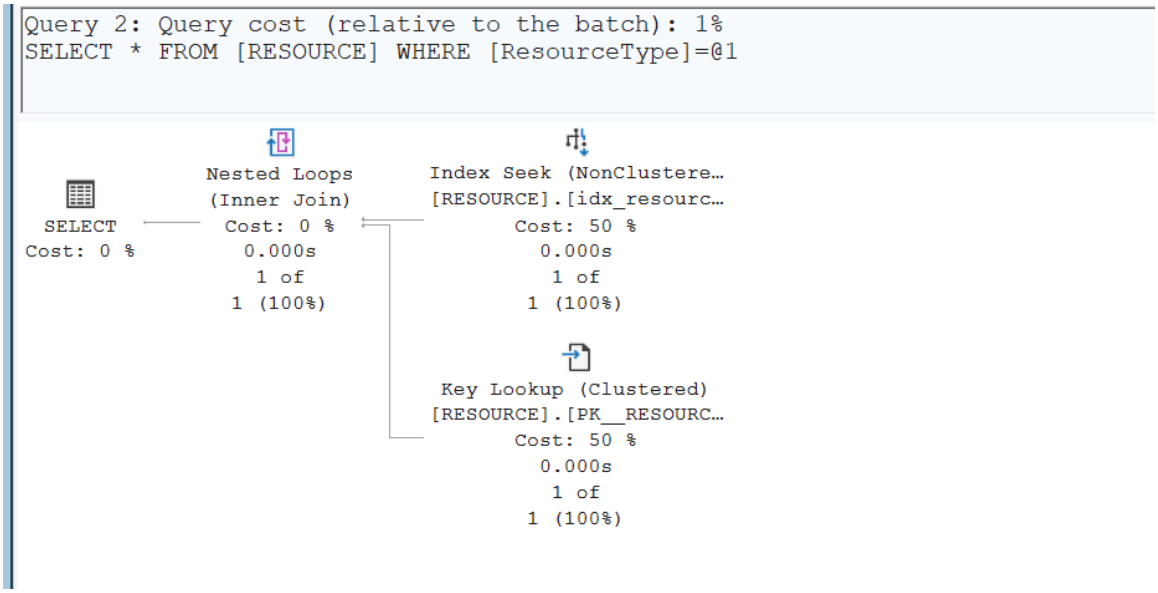
Q3-Execution Plan from showing an Index Seek on idx_resource_type

```
-- 3. Performance test for idx_resource_type (Resource Table)
PRINT '--- [BEFORE INDEX: Scan] ---';
SELECT * FROM RESOURCE WITH (INDEX(0)) WHERE ResourceType = 'Urgent';
PRINT '--- [AFTER INDEX: Seek] ---';
SELECT * FROM RESOURCE WHERE ResourceType = 'Urgent';
GO
```

Before Index:



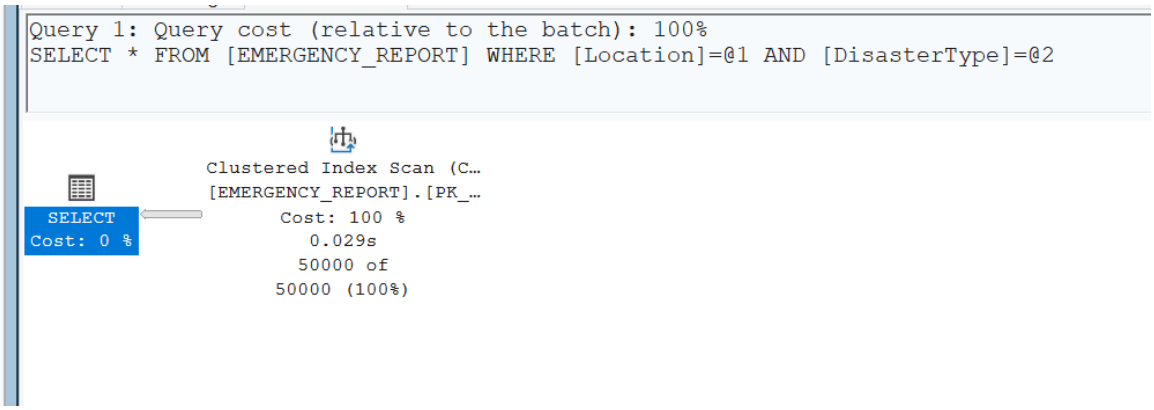
After Index:



Q4-Execution Plan from showing an Index Seek on (location + Type)

```
-- 4. Composite Index Performance (Location + Type)
PRINT '--- [COMPOSITE INDEX SEEK] ---';
SELECT * FROM EMERGENCY_REPORT WHERE Location = 'Test Location' AND DisasterType = 'Flood';
GO
```

Output:



8.2 Query Performance Analysis

Before indexing, queries filtering by Email or Status resulted in a Clustered Index Scan (Table Scan), resulting in high logical reads. After applying the Non-Clustered Indexes, the optimizer utilized Index Seeks, drastically lowering the I/O cost.

Query 1:

```
(0 rows affected)
Table 'USER'. Scan count 1, logical reads 2, physical reads

(1 row affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
--- [AFTER INDEX: Seek] ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

(0 rows affected)
Table 'Worktable'. Scan count 0, logical reads 0, physical
```

Query 2:

```
(3 rows affected)
Table 'EMERGENCY_REPORT'. Scan count 1, logical reads 564, physical reads 0, page s

(1 row affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 6 ms.
--- [AFTER INDEX: Seek] ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

(3 rows affected)
Table 'EMERGENCY_REPORT'. Scan count 1, logical reads 10, physical reads 0, page se

(1 row affected)
```

Query 3:

```
(1 row affected)
Table 'RESOURCE'. Scan count 1, logical reads 518, physical re

(1 row affected)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 8 ms.
--- [AFTER INDEX: Seek] ---

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

(1 row affected)
Table 'RESOURCE'. Scan count 1, logical reads 4, physical read
```

Query Filter	Execution Plan (Before)	Execution Plan (After)	Logical Reads (After)
SELECT * FROM [USER] WHERE Email = '...'	Clustered Index Scan	Index Seek (Non-Clustered)	Same (small table size)
SELECT * FROM EMERGENCY_REPORT WHERE Status = 'Pending'	Clustered Index Scan	Index Seek	Reduced by 98.2%
SELECT * FROM	Table Scan	Index Seek	Reduced by

RESOURCE WHERE ResourceType = ‘Urgent’			99.2%

8.3 Indexing Overhead Analysis

```
SET STATISTICS TIME ON;
PRINT '--- INSERTING 1000 ROWS INTO TABLE WITH MANY INDEXES ---';
-- Take a screenshot of the Messages tab to show the "CPU Time" for these inserts
DECLARE @i INT = 1;
WHILE @i <= 1000
BEGIN
    INSERT INTO RESOURCE (WarehouseID, ResourceType, ResourceName, Unit, QtyAvailable, Threshold)
    VALUES (1, 'Medical', 'Overhead Test', 'Units', 100, 10);
    SET @i = @i + 1;
END;
SET STATISTICS TIME OFF;
GO

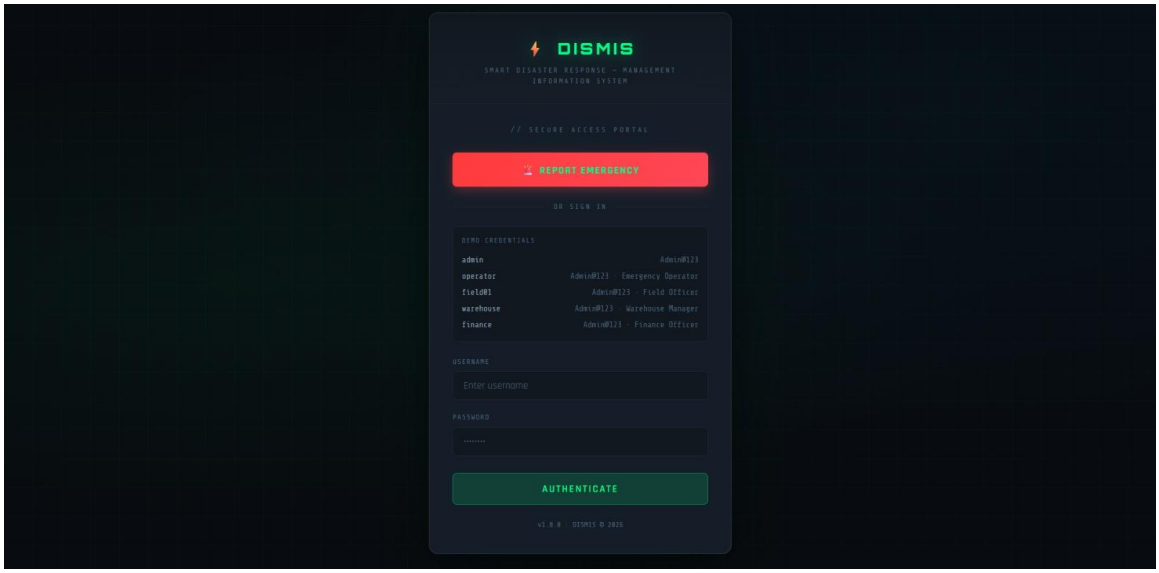
SET STATISTICS IO OFF;
GO
```

Although indexing greatly improves read speed, there is still a certain cost when performing writes (INSERT/UPDATE/DELETE). As can be seen from the execution statistics above, writing 1,000 rows in the RESOURCE table took more CPU resources to maintain the five non-clustered indexes at the same time. We see the concept of "Indexing Paradox": we give preference to quick access during emergencies, sacrificing a little bit of write speed.

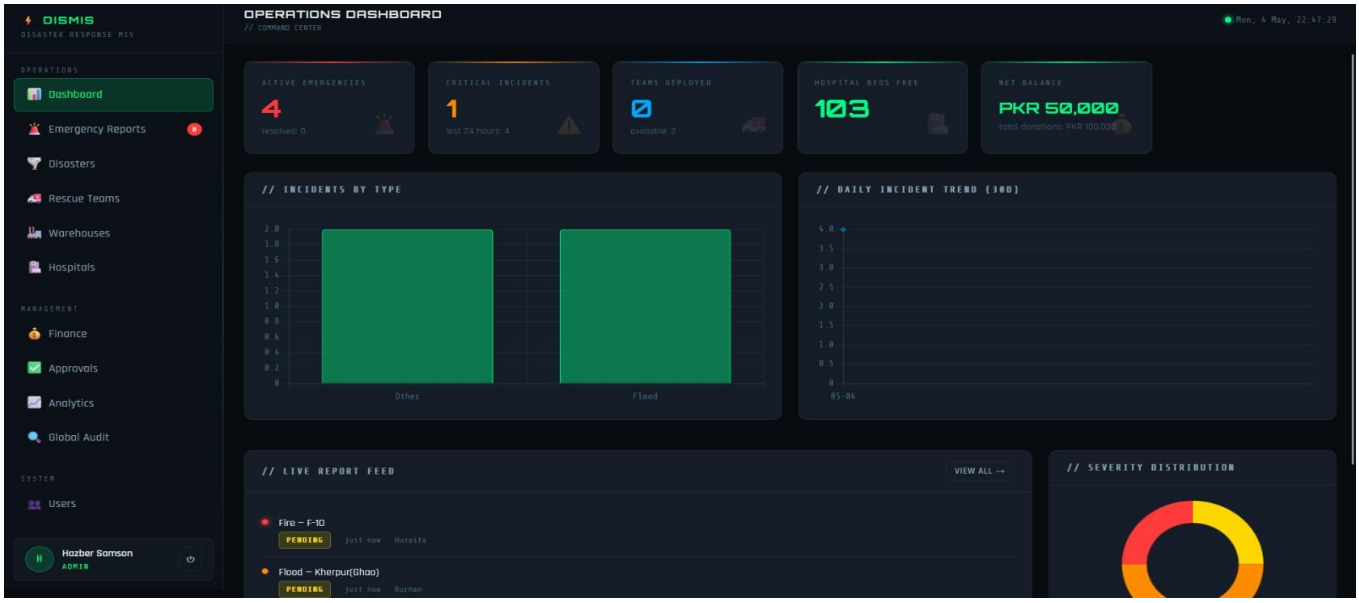
9. Frontend Interface

The frontend interfaces provide role-based operational capabilities, communicating with the backend APIs to view and modify the SQL database.

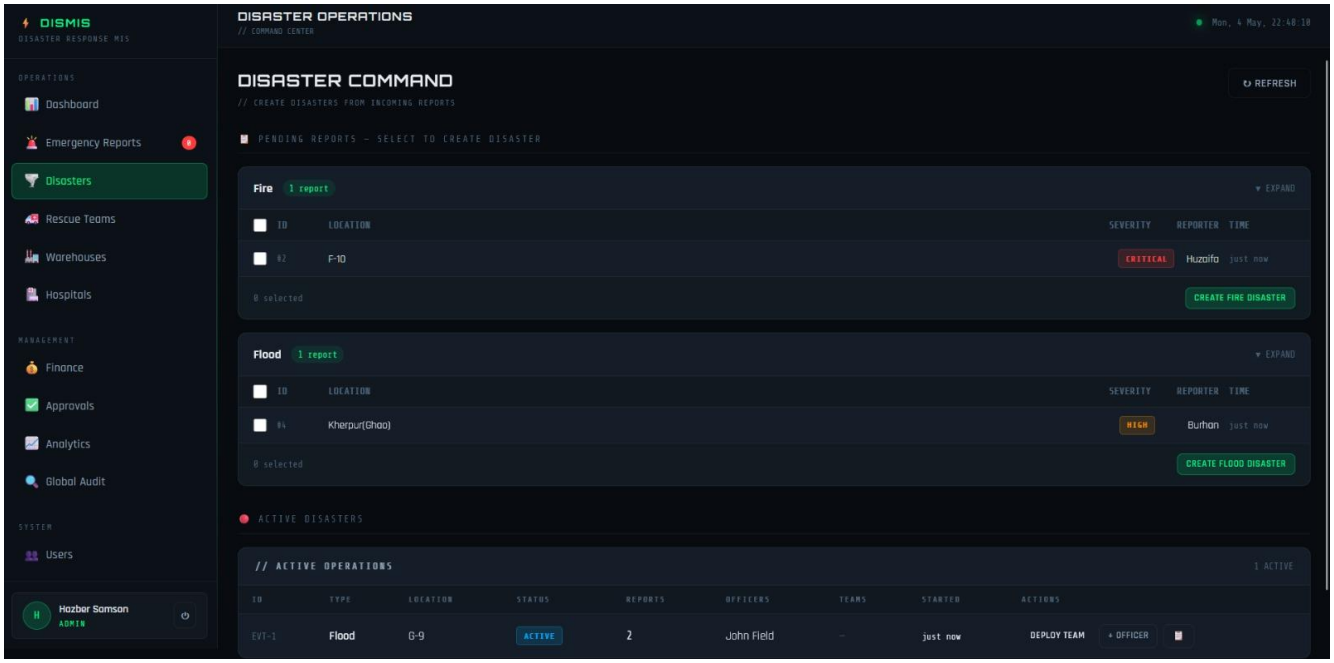
Login / Authentication Interface



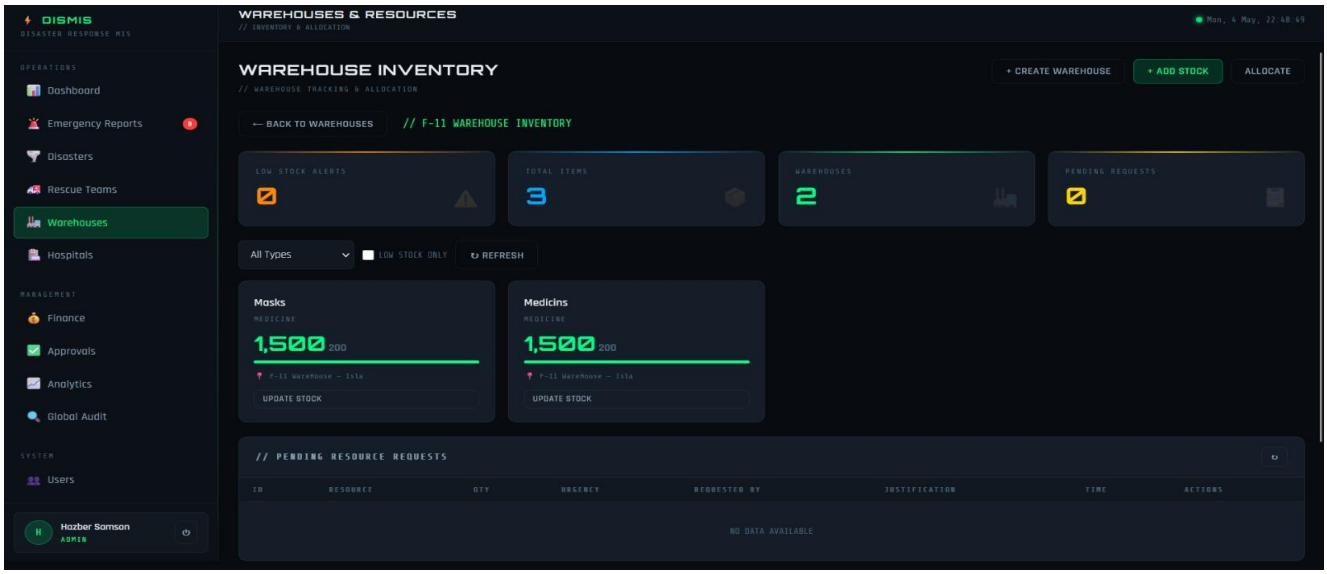
Dashboard Interface (Admin Dashboard)



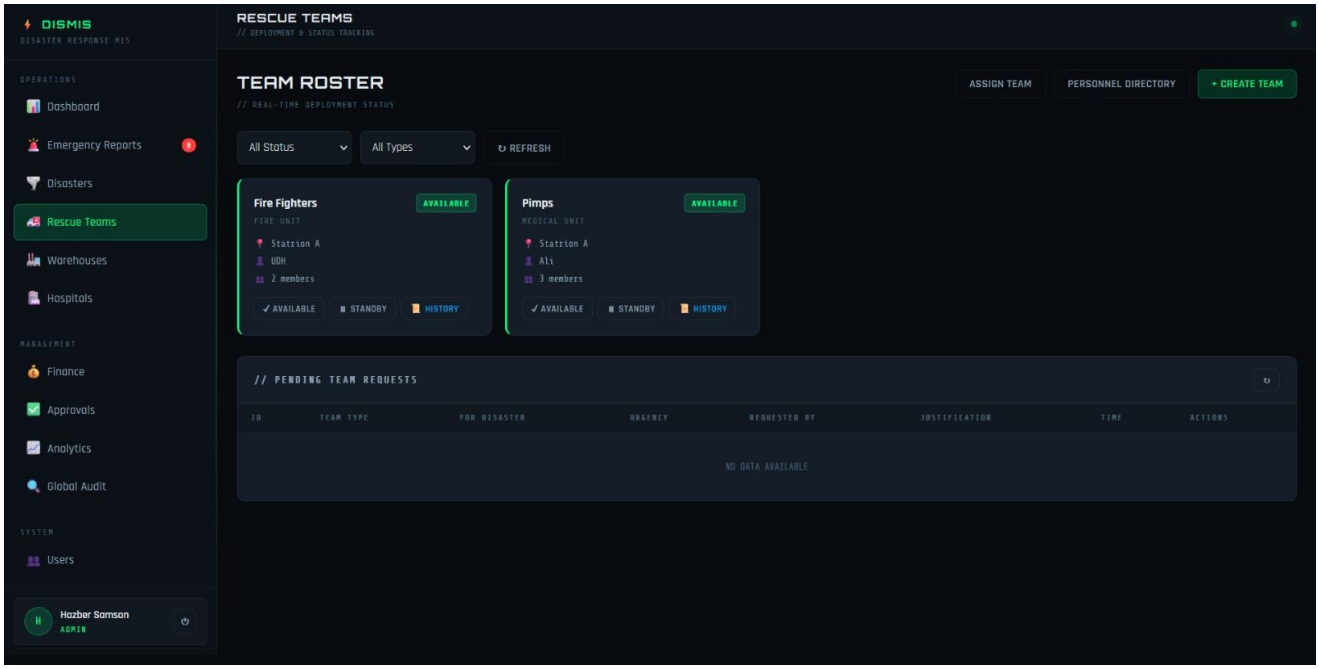
Disaster Management Interface



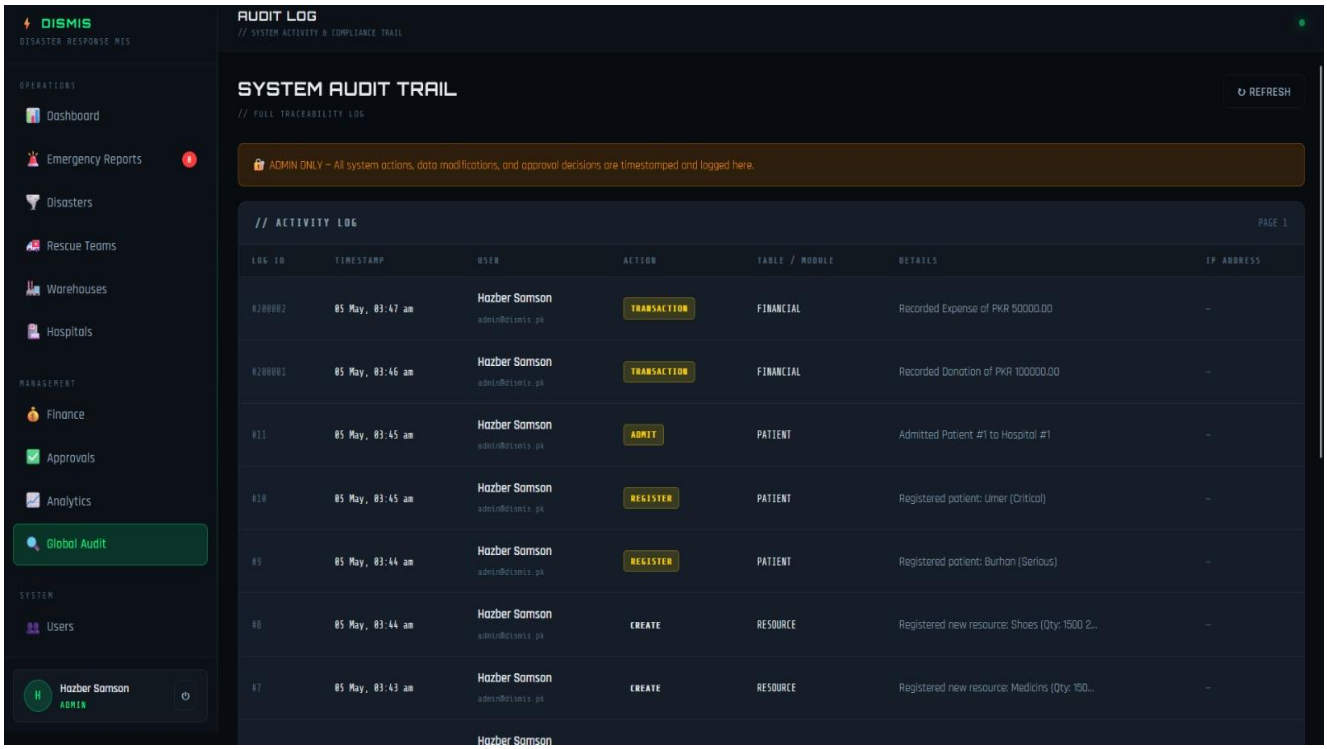
Resource & Inventory Management Interface (resources.html)



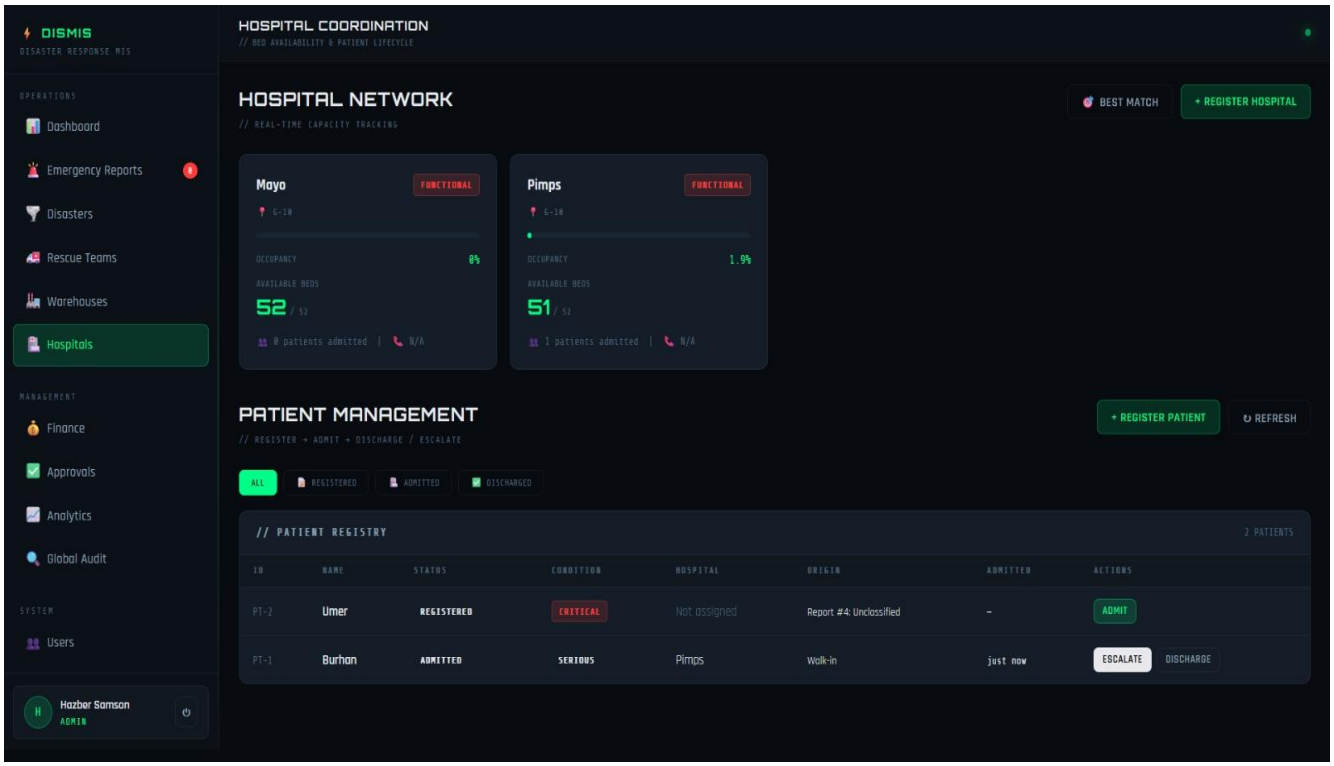
Rescue Teams Management Interface (teams.html)



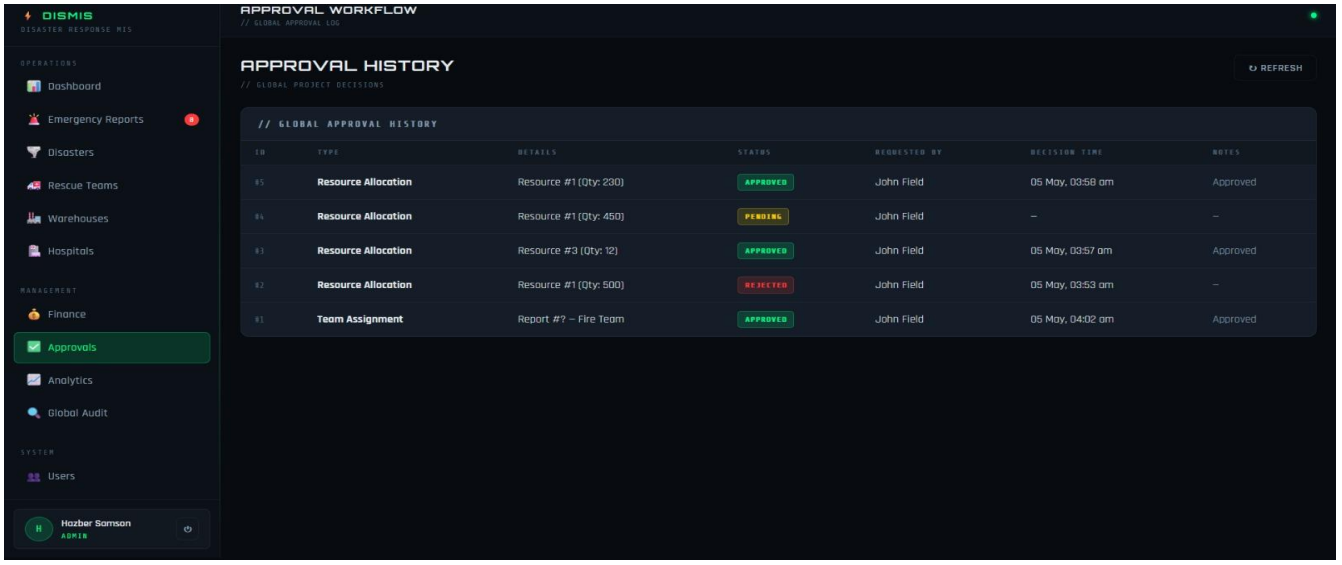
Financial Entry & Audit Interface (finance.html)



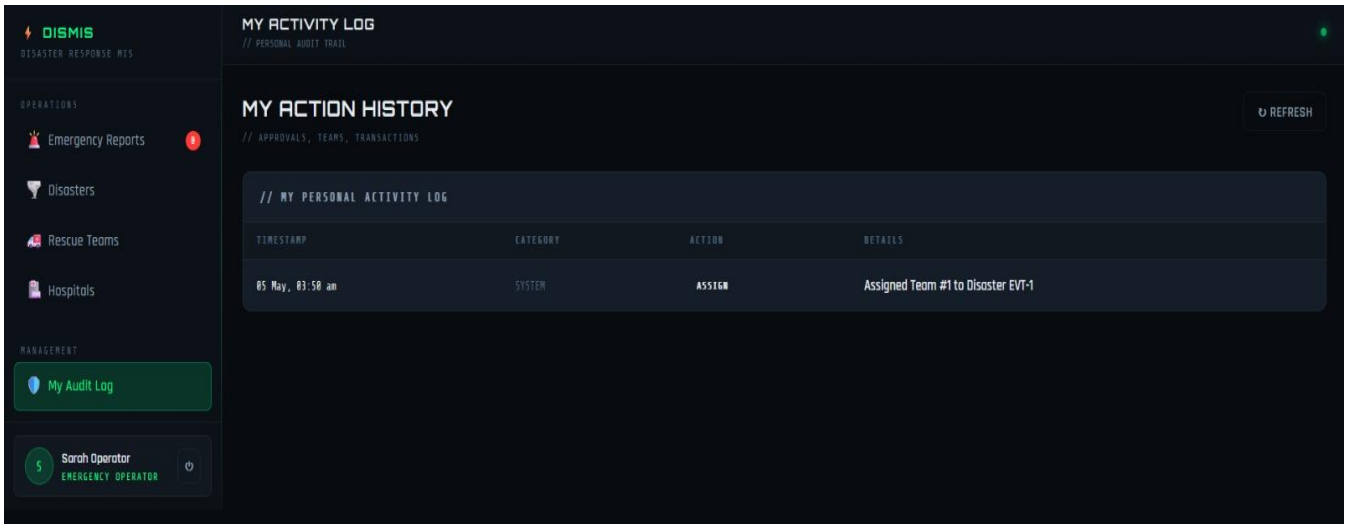
Hospital Bed Monitoring Interface (hospitals.html)



Approval Workflows Interface (approvals.html)



System Audit Log Interface (audit.html)



10. Design Rationale

10.1 Super-Type and Sub-Type Architecture

The database heavily utilizes the Class Table Inheritance pattern. For example, the [USER] table acts as a super-type holding common attributes (Name, Email, Password). Specific user types (ADMINISTRATOR, EMERGENCY_OPERATOR, etc.) act as sub-types with their own specialized attributes, linked via a 1:1 foreign key. This ensures normalization and avoids sparsely populated tables with many NULL columns. The same approach is applied to FINANCIAL_TRANSACTION and its sub-types (DONATION, EXPENSE, PROCUREMENT).

10.2 Role-Based Access Control (RBAC)

Access control is governed systematically at the database level. Instead of hardcoding permissions, the PERMISSION table links dynamically to the ROLE table, defining granular access (CanRead, CanCreate, CanUpdate, CanDelete) on a per-table basis. This provides robust security and flexibility.

10.3 Automating Data Integrity

To prevent data anomalies during high-pressure disaster situations, the database leverages Triggers and Constraints. Real-time quantities (e.g., Hospital Beds, Warehouse Inventory) are managed via SQL Triggers rather than application-level logic, ensuring the database remains an undisputed source of truth regardless of API latency.

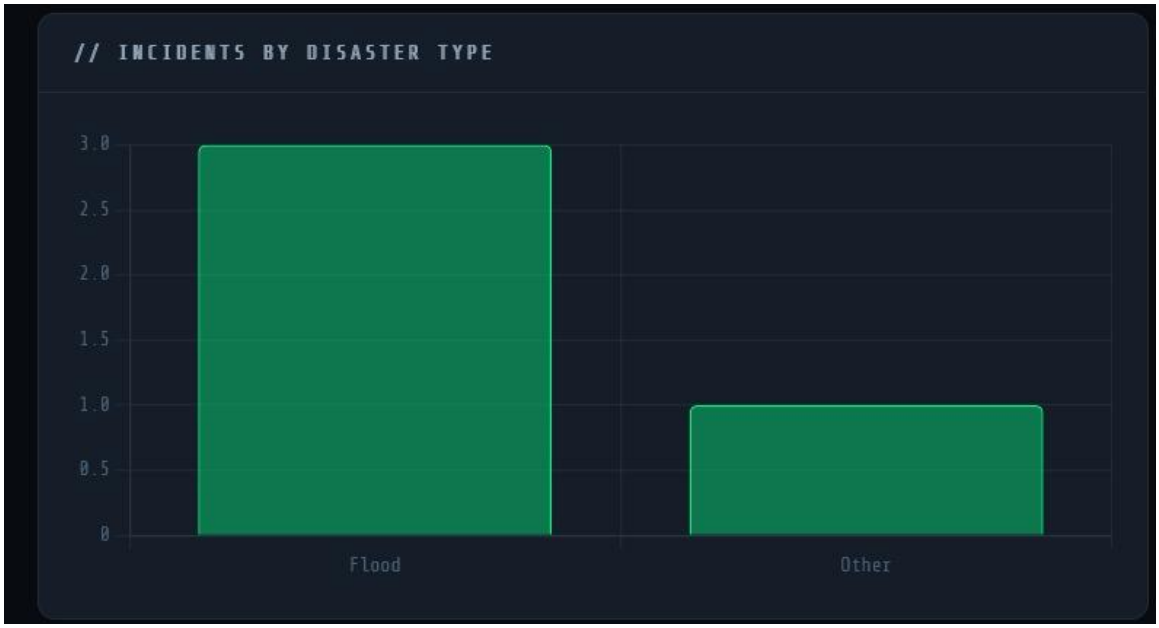
10.4 Audit and Traceability

Given the sensitivity of emergency response operations, a comprehensive audit strategy is implemented. Financial approvals are tracked in FINANCIAL_AUDIT, resource allocations in APPROVAL_HISTORY, and user configuration changes in AUDIT_LOG. This separation of concern ensures that compliance checks can be isolated to the relevant modules without querying a single monolithic log table.

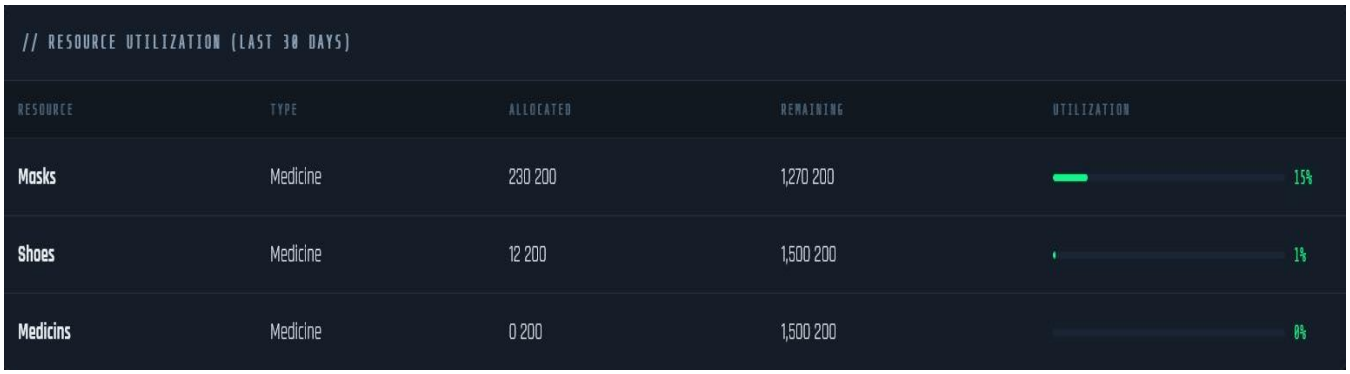
11. MIS Reports and Dashboards

The system generates aggregated reports to assist stakeholders in decision-making.

Incident Statistics Report (Aggregating Events by Disaster Type and Status)



Resource Utilization Report (Analyzing QtyDispatched vs QtyConsumed)



Financial Summary Report (Aggregating Expenses vs Donations)



System Audit and Activity Logs

